

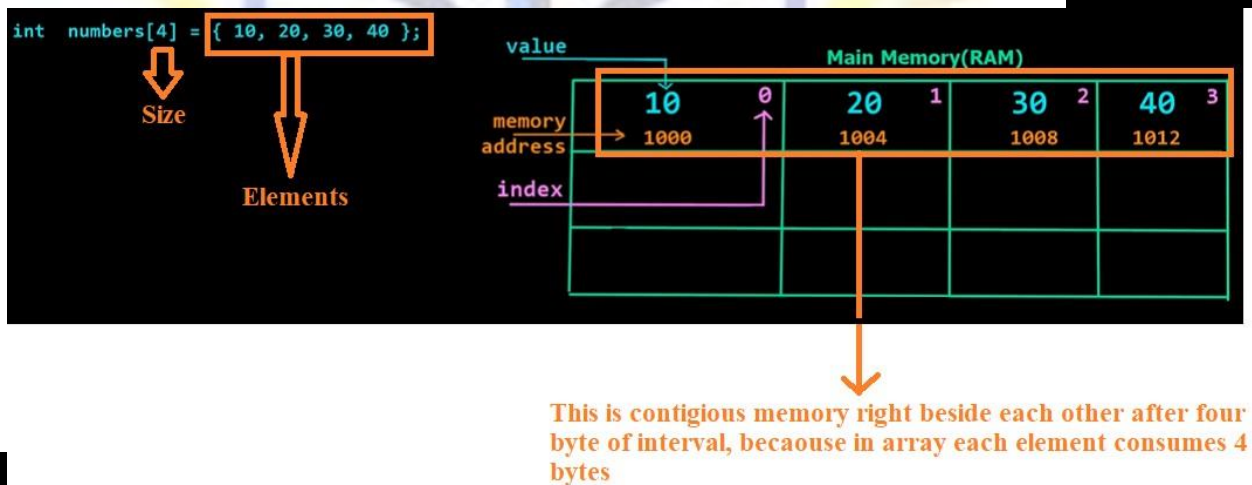
Data Structures and Algorithms (CC-213)

https://youtube.com/playlist?list=PLIY8eNdw5tW_zX3OCzX7NJ8bL1p6pWfqG&si=7plhpYV21wJwTKNN

Link of DSA playlist On Youtube

Arrays

An array is a collection of elements of the same type placed in contiguous memory locations that can be individually referred to by using an index or a unique identifier.



ABSTRACT DATA TYPE (ADT)

There are two ways to study DS. One is *ABSTRACT / LOGICAL* view and other is *IMPLEMENTATION* view

What is Abstract Data Type ?

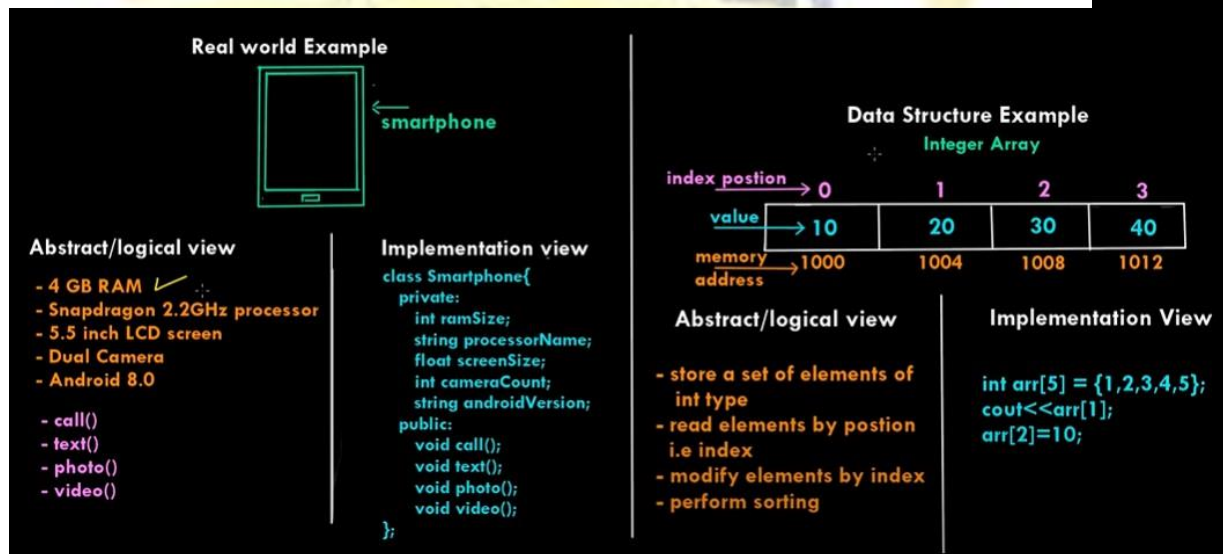
Data Structures Definition : In computer science, a data structure is a **data organization, management and storage** format that enables **efficient access and modification**.

2 Ways of looking at Data Structures –

- Mathematical/Logical/Abstract Models/Views
- Implementation

What is Abstract Data Type ?

Definition : ADTs are entities that are definitions of data and operations but do not have implementation details.



ALGORITHM

What is an Algorithm ?

Dictionary Definition : A **process** or **set of rules** to be followed in calculations or other problem-solving operations, especially by a computer.

Formal Definition : An algorithm is a **finite set of instructions** that are carried in a specific order to perform **specific task**.

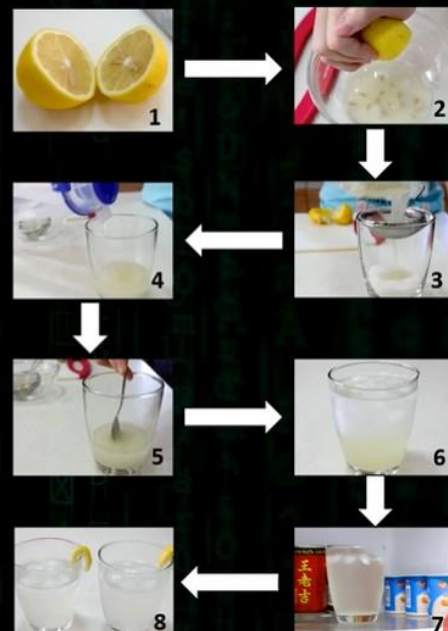
Algorithms typically have the following characteristics –

- **Inputs** : 0 or more input values.
- **Outputs** : 1 or more than 1 output.
- **Unambiguity** : clear and simple instructions.
- **Finiteness** : Limited number of instructions.
- **Effectiveness** : Each instruction has an impact on the overall process.

Real World example of an Algorithm –

Algorithm(aka process) to make a lemonade –

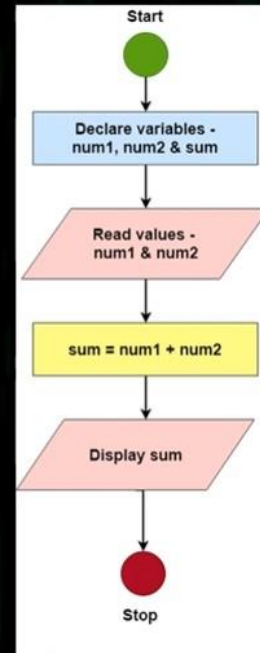
1. Cut your lemon in half.
2. Squeeze all the juice out of it that you can.
3. Pour your juice into a container with 1/4 cup (2 oz) sugar.
4. Add a very small amount of water to your container.
5. Stir your solution until sugar dissolves.
6. Fill up container with water and add ice.
7. Put your lemonade in the fridge for five minutes.
8. Serve and enjoy!



Example of an Algorithm in Programming –

Write an algorithm to add two numbers entered by user. –

1. Step 1: Start
2. Step 2: Declare variables num1, num2 and sum.
3. Step 3: Read values num1 and num2.
4. Step 4: Add num1 and num2 and assign the result to sum. (sum $\leftarrow$ num1 + num2)
5. Step 5: Display sum
6. Step 6: Stop



STACK

What is Stack Data Structure ?

Definition : *Stack is a linear data structure which operates in a LIFO (Last In First Out) or FILO (First In Last Out) pattern.*

- It is named stack as it behaves like a real-world stack, for example – a deck of cards or a pile of plates, etc.
- Stack is an abstract data type with a bounded (predefined) capacity.
- It is a simple data structure that allows adding and removing elements in a particular order.
- The order may be LIFO (Last In First Out) or FILO (First In Last Out).



Working of Stack -

Stack Data Structure operates in a LIFO (Last In First Out) pattern or FILO (First In Last Out) pattern.

- >> Items are added on top of the stack. This is known as PUSH operation
- >> Items are removed from top of the stack. This is known as POP operation

Standard Stack Operations -

- 1) **push()** -
Place an item onto the stack. If there is no place for new item, stack is in overflow state.
- 2) **pop()** -
Return the item at the top of the stack and then remove it.
If pop is called when stack is empty, it is in an underflow state.
- 3) **isEmpty()** -
Tells if the stack is empty or not
- 4) **isfull()** -
Tells if the stack is full or not.
- 5) **peek()** -
Access the item at the i position
- 6) **count()** -
Get the number of items in the stack.
- 7) **change()** -
Change the item at the i position
- 8) **display()** -
Display all items in the stack

Some Applications of Stack Data Structure

- **Balancing** of symbols
- **Infix** to **Postfix /Prefix** conversion
- **Redo-undo** features at many places like editors, **photoshop**.
- **Forward** and **backward** feature in web browsers
- Used in many algorithms like Tower of Hanoi, tree traversals, stock span problem, histogram problem.
- Other applications can be Backtracking, Knight tour problem, rat in a maze, N queen problem and sudoku solver
- In Graph Algorithms like Topological Sorting and Strongly Connected Components

EXAMPLE OF STACK & IT'S OPERATIONS

```
#include <iostream>
#include <string>

using namespace std;

class Stack{
    private:
        int top;
        int arr[5];

    public:
        Stack()
        {
            top = -1;
            for(int i = 0;i<5;i++)
            {
                arr[i] = 0;
            }
        }

        bool isEmpty()
        {
            if(top== -1)
                return true;
            else
                return false;
        }

        bool isFull()
        {
            if(top == 4)
                return true;
            else
                return false;
        }
}
```

```

    }
void push(int val)
{ if(isFull()) {
    cout<<"stack overflow"<<endl;
    }
    else
    {
        top++;
        arr[top] = val;
    }
}
int pop()
{
    if(isEmpty())
    {
        cout<<"Stack underflow"<<endl;
        return 0;
    }
    else
    {
        int popValue = arr[top];
        arr[top] = 0;
        top--;
        return popValue;
    }
}
int count() {
    return(top+1);
}
int peek(int pos) {
    if(isEmpty())
    {
        cout<<"Stack underflow"<<endl;

```



```

        return 0;
    }
    else
    {
        return arr[pos];
    }
}

```

```

void change(int val, int pos)

```

```

{
    arr[pos] = val;
    cout<<"value changed at location "<< pos<<endl;
}

```

```

void display() {

```

```

    cout<<"all values in the stack are: "<<endl;
    for(int i = 4; i >= 0; i--)
    {
        cout<<arr[i]<<endl;
    }
}

```

```

};

```

```

int main()

```

```

{

```

```

    Stack s1;

```

```

    int option, position, value;

```

```

    do
    {

```

```

        cout<<"what operation do you want to perform"<<endl;

```

```

        cout<<"1. PUSH() "<<endl;

```

```

        cout<<"2. POP() "<<endl;

```

```

        cout<<"3. isEmpty() "<<endl;
    }
}

```

```

cout<<"4. isFull() "<<endl;
cout<<"5. peek() "<<endl;
cout<<"6. count() "<<endl;
cout<<"7. change() "<<endl;
cout<<"8. display() "<<endl;
cout<<"9. clear screen"<<endl<<endl;

cin>>option;

switch(option)
{
case 1:
    cout<<"enter an item to push in the stack"<<endl;
    char ch;
    do{
        ch>>
    }while(ch!='\n');
    s1.push(ch);
break;
case 2:
    cout<<"pop function is called - popped vale: "<<s1.pop()<<endl;
    break;
case 3:
    if(s1.isEmpty()){
        cout<<"stack is empty"<<endl;
        break;
    }else
        cout<<"stack is not empty"<<endl;
    break;
case 4:
    if(s1.isFull()){
        cout<<"stack is FULL"<<endl;
        break;
    }else
        cout<<"stack is not full"<<endl;
    break;
case 5:

```

```
    cout<<"enter position of item you want to peek: "<<endl;
    cin>>position;
    cout<<"peak function called - value at position "<<position<<" is
"<<s1.peek(position)<<endl;
    break;
case 6:
    cout<<"count function called - "<<"number of items in the stack are
"<<s1.count()<<endl;
    break;
case 7:
    cout<<"change function called - ";
    cout<<"enter position of item you want to change: ";
    cin>>position;
    cout<<endl;
    cout<<"enter value of item you want to change: ";
    cin>>value;
    s1.change(position, value);
    break;
case 8:
    cout<<"display function called - "<<endl;
```



```
        s1.display();  
        break;  
case 9:  
    system("cls");  
    break;  
default:  
    cout<<"enter proper option number"<<endl;  
    }  
  
}while(option!=0);  
    return 0;  
}
```



QUEUE

What is Queue Data Structure ?

Definition : Queue is a *linear data structure* which operates in a *First IN First OUT* or *Last IN Last OUT*.

- It is named queue as it behaves like a real-world queue, for example – queue(line) of cars in a single lane, queue of people waiting at food counter etc.
- Queue is an *abstract data type* with a bounded (predefined) capacity.
- It is a simple data structure that allows adding and removing elements in a particular order.
- The order is *FIFO(First IN First OUT)* or *LIFO(Last In Last Out)*.



Working of Queue -

Queue is a linear data structure which operates in a First IN First OUT or Last IN Last OUT.

>> Elements are added at one end(rear end). This is called Enqueue

>> Elements are removed from the other end(front/head end). This is called Dequeue



Dequeue

starts as 0, 1, 2, 3

front ++

value goes from
front end to rear
end

Enqueue

starts as 0, 1, 2, 3

rear ++

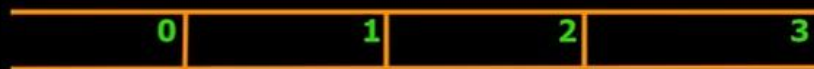
values go from rear
end to front end



in enqueue operation elements are added from rear end. and in dequeue operation elements are removed from front end

Initialization -

```
int arr[4];
int rear = -1
int front = -1
```



operations

```
isEmpty()
{
    if(front == -1 && rear == -1)
        return true
    else
        return false
}

isFull()
{
    if(rear == size(arr)-1)
        return true;
    else
        return false;
}
```

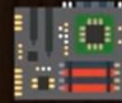
```
enqueue(value)
{
    if(isFull())
        return;
    else if(isEmpty())
    {
        rear = front = 0
    }
    else
    {
        rear++
    }
    arr[rear] = value
}
```

```
dequeue()
{
    int x = 0;
    if(isEmpty())
        return;
    else if(front == rear)
    {
        x = arr[front]
        front = rear = -1
    }
    else
    {
        x = arr[front]
        front ++
    }
    return x;
}
```

Some Applications of Queue Data Structure

Queue is used when things but have to be processed in **First In First Out** order. Like –

- CPU scheduling, Disk Scheduling.
- Handling of interrupts in **real-time** systems. The interrupts are handled in the same order as they arrive, First come first served.
- In real life, **Call Center** phone systems will use Queues, to hold people calling them in an order, until a service representative is free.
- When data is transferred **asynchronously** between two processes. Queue is used for synchronization.



Example of queue and its operations

```
#include <iostream>

using namespace std;

class Queue{
private:
    int arr [5];
    int rear;
    int front;

public:
    Queue ()
    {
        front = -1;
        rear = -1;
        for(int i = 0;i<5;i++)
        {
            arr[i] = 0;
        }
    }

    bool isEmpty(){
        if(rear == -1 && front == -1)
            return true;
        else
            return false;
    }

    bool isFull(){
        if(rear == 4 && front == 0)
            return true;
        else
            return false;
    }
}
```

```
}
```

```
void enqueue(int val){
```

```
    if isFull
```

```
        <<"Queue is full"<<
```

```
    else
```

```
        if isEmpty
```

```
            = 0 & front
```

```
            = 0
```

```
            arr[rear] = val;
```

```
        }
```

```
    else{
```

```
        rear++;
```

```
        arr[rear] = val;
```

```
    }
```

```
}
```

```
int dequeue(){
```

```
    int x;
```

```
    if(isEmpty()){
```

```
        cout <<"queue nis empty"<<endl;
```

```
        return 0;
```

```
    }else if (front == rear){
```

```
        x = arr[front];
```

```
        arr[front] = 0;
```

```
        rear = -1;
```

```
        front = -1;
```

```
        return x;
```

```
    }else{
```

```
        x = arr[front];
```

```
        arr[front] = 0;
```

```
        front++;  
        return x;  
    }  
}
```

```
int count(){  
    return (rear-front+1);  
}
```

```
void display(){  
    cout<<"all values in the queue are: "<<endl;  
    for(int i =0;i<5;i++){  
        cout<<arr[i]<<" ";  
    }
```

```
int main()  
{  
    Queue q1;  
    int option, value;  
    do  
    {  
        cout<<"\n\nWhat operation do you want to perform select an  
option"<<endl;  
        cout<<"1. Enqueue() "<<endl;  
        cout<<"2. Dequeue() "<<endl;  
        cout<<"3. isEmpty() "<<endl;  
        cout<<"4. isFull() "<<endl;  
        cout<<"5. count() "<<endl;
```

```
cout<<"6. display()"<<endl;
cout<<"7. clear screen"<<endl<<endl;
```

```
cin>>option;
```

```
switch(option)
```

```
{
```

```
    case 0:
```

```
        break;
```

```
    case 1:
```

```
        cout<<"enter an value to enqueue"<<endl;
```

```
        cin>>value;
```

```
        q1.enqueue(value);
```

```
        break;
```

```
    case 2:
```

```
        cout<<"dequeued value: "<<q1.dequeue()<<endl;
```

```
        break;
```

```
    case 3:
```

```
        if q1.isEmpty() cout<<"queue
```

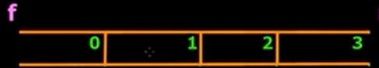
```
is empty"<<endl;
```



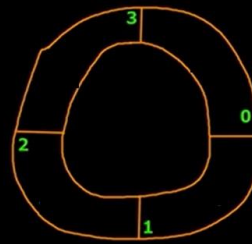
```
        else cout<<"queue is not  
        empty"<<endl; break;  
  
        case 4:  
        if(q1.isFull())  
            cout<<"queue is full"<<endl;  
            else  
                cout<<"queue is not full"<<endl;  
        break;  
  
        case 5:  
        cout<<"queue size is : "<<q1.count()<<endl;  
        break;  
  
        case 6:  
        cout<<"display function called"<<endl;  
        q1.display();  
        break;  
  
        case 7:  
        system("cls");  
        break;  
  
        default:  
        cout<<"invalid option. please select a valid option"<<endl;  
    }  
} while(option!=0);  
  
return 0;  
}
```

CIRCULAR QUEUE DATA STRUCTURE

Initialization -
 int arr[4];
 int rear = -1
 int front = -1



```
enqueue(value)
{
    if(isFull())
        return;
    else if(isEmpty)
    {
        rear = front = 0;
    }
    else
    {
        // in simple queue function
        // it was rear ++
        rear = (rear+1)%N;
    }
    arr[rear] = value;
}
```



$$\begin{array}{r} 4 \overline{) 3} \\ \underline{-0} \\ 3 \end{array} \quad \begin{array}{r} 4 \overline{) 4} \\ \underline{-4} \\ 0 \end{array}$$

size of array

when r points to index 2 so $2+1=3$ $3\%4=3$, as % always give us the reminder. so when w do one more enqueue operation now r will mpoints index 3 and $3+1=4$, $4\%4=0$. so the main benefit of circular queue is that rear pointer can again points towards index 0 irrespective of front pointer



Initialization -
int arr[4];
int rear = -1
int front = -1

f
0

```
enqueue(value)
{
    if(isFull())
        return;
    else if(isEmpty())
    {
        rear = front = 0;
    }
    else
    {
        rear = (rear+1)%N;
    }
    arr[rear] = value;
}

isEmpty()
{
    if(front == -1 && rear == -1)
        return true;
    else
        return false;
}

isFull()
{
    if((rear+1)%N == front)
        return true;
    else
        return false;
}
```

EXAMPLE OF CIRCULAR QUEUE

```
#include <iostream>
using namespace std;

class CircularQueue{
private:
    int arr [5];
    int rear;
    int front;
    int itemCount; // THIS IS CHANGE *

public:
    CircularQueue()
    {
        itemCount = 0; // part of change *
        front = -1;
        rear = -1;
        for(int i = 0; i < 5; i++)
        {
```

```
        arr[i] = 0;
    }
}
```

```
bool isEmpty(){
    if(rear == -1 && front == -1)
        return true;
    else
        return false;
}
```

```
bool isFull(){
    if((rear+1)%5==front) // this is the basic change in circular queue
from simple queue
    return true;
    else
        return false;
}
```

```
void enqueue int val {
```



```

    if(isFull()){
        cout<<"Queue is full"<<endl;
    }
    else if(isEmpty()){
        rear = 0;
        front = 0;
        arr[rear] = val;
        itemCount++; // this is change *
    }
    else{
        rear = (rear+1)%5; // this is the second change
        arr[rear] = val;
        itemCount++; // this is change *
    }
}

int dequeue(){
    int x;
    if(isEmpty()){
        cout <<"queue nis empty"<<endl;
        return 0;
    }else if (front == rear){
        x = arr[front];
        arr[front] = 0;
        rear = -1;
        front = -1;
        itemCount--; // this is change *
        return x;
    }else{
        x = arr[front];
        arr[front] = 0;
        front = (front+1)%5; // this is the third change
    }
}

```

```

        itemCount--; // this is change *
        return x;
    }
}

int count
    return
}
void display
    cout<<"all values in the queue are: "<<endl;
    for(int i =0;i<5;i++){
        cout<<arr[i]<<" ";
    }
}
};

int main()
{
    CircularQueue q1;
    int option, value;

    do
    {
        cout<<"\n\nWhat operation do you want to perform select an
option"<<endl;
        cout<<"1. Enqueue () "<<endl;
        cout<<"2. Dequeue () "<<endl;
        cout<<"3. isEmpty () "<<endl;
        cout<<"4. isFull () "<<endl;
        cout<<"5. count () "<<endl;
        cout<<"6. display () "<<endl;

```

```
cout<<"7. clear screen"<<endl<<endl;
```

```
cin>>option;
```

```
switch(option)
```

```
{
```

```
    case 0:
```

```
        break;
```

```
    case 1:
```

```
        cout<<"enter an value to enqueue"<<endl;
```

```
        cin>>value;
```

```
        q1.enqueue(value);
```

```
        break;
```

```
    case 2: cout<<"dequeued value:"
```

```
        "<<q1.dequeue()<<endl; break;
```

```
    case 3:
```

```
        if(q1.isEmpty())
```

```
            cout<<"queue is empty"<<endl;
```

```
        else
```

```
            cout<<"queue is not empty"<<endl;
```

```
        break;
```

```
    case 4:
```

```
        if(q1.isFull())
```

```
            cout<<"queue is full"<<endl;
```

```
        else
```

```
            cout<<"queue is not full"<<endl;
```

```
        break;
```

```
    case 5:
```

```
        cout<<"queue size is : "<<q1.count()<<endl;
```

```
        break;
```



```
case 6:
    cout<<"display function called"<<endl;
    q1.display();
    break;

case 7:
    system("cls");
    break;

default:
    cout<<"invalid option. please select a valid option"<<endl;
}
} while(option!=0);

return 0;
}
```

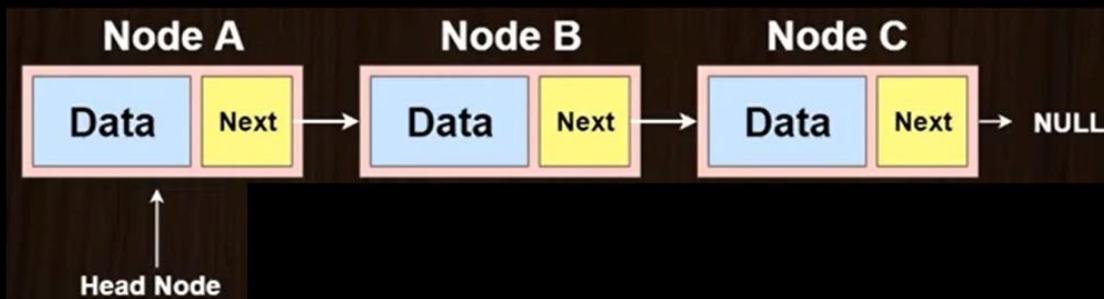


LINKED LIST DATA STRUCTURE

What is Linked List Data Structure ?

Definition : A *linked list* is a *linear data structure*, in which the elements are **not** stored at **contiguous** memory locations. The elements in a linked list are **linked using pointers**(entity that point to the next element)

In simple words, a linked list consists of **nodes** where each **node** contains a **data field** and a **reference(link)** to the next node in the list.



Working of Linked List -

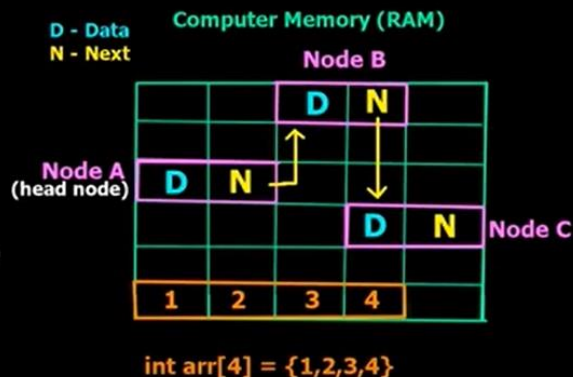
Linked list consists of nodes where each node contains a data field and a reference(link) to the next node in the list.

1) Node/Link/Element/Object -

Each node in the linked list consists of two parts -

- a) data
- b) link to the next node

2) Next - This points to the next node/element in the linked list (since they are not stored in a contiguous memory locations)



LINKED LIST VS ARRAY

Linked List vs Arrays

Advantages of Linked List over Arrays -

- 1) Dynamic size
- 2) Ease of insertion/deletion

Disadvantages of Linked List over Arrays -

- 1) Random access is not allowed. We have to access elements sequentially starting from the first node.
- 2) Extra memory space for a pointer is required with each element of the list.
- 3) Not cache friendly. Since array elements are contiguous locations, there is locality of reference which is not there in case of linked lists.

#01	#02	#03	#04	#05	#06
1 ⁰	3 ¹	5 ²		4	5

int arr[6] = {1,3,5}



OPERATION OF LINKED LIST

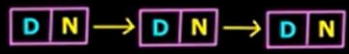
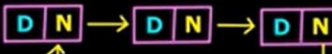
Operations of Linked List -

1. Traversing a linked list. (head) <pre>graph LR head((head)) --> node1[D N] node1 --> node2[D N] node2 --> node3[D N]</pre>	2. Append a new node (to the end) of a list (head) <pre>graph LR head((head)) --> node1[D N] node1 --> node2[D N] node2 --> node3[D N]</pre>	3. Prepend a new node (to start) of list (head) <pre>graph LR head((head)) --> node1[D N] node1 --> node2[D N] node2 --> node3[D N]</pre>
4. Inserting a new node to a specific position in the list (head) <pre>graph LR head((head)) --> node1[D N] node1 --> node2[D N]</pre>	5. Deleting a node from the list (head) <pre>graph LR head((head)) --> node1[D N] node1 --> node2[D N] node2 --> node3[D N]</pre>	6. Updating a node in the list (head) <pre>graph LR head((head)) --> node1[D N] node1 --> node2[D N] node2 --> node3[D N]</pre>

TYPES OF LINKED LIST

Types of Linked List -

D = Data P = Previous N = Next

1) Singly Linked List	2) Doubly Linked List	3) Circular Linked List
(head) 	(head) 	(head) 

Some Applications of Linked List Data Structure

Some Applications of Linked List are as follows –

- Linked Lists can be used to implement **Stacks** , **Queues**.
- Linked Lists can also be used to implement **Graphs**. (Adjacency list representation of Graph).
- Implementing Hash Tables :- Each Bucket of the hash table can itself be a linked list. (Open chain hashing).
- **Undo** functionality in Photoshop or Word . Linked list of states



SINGLY LINKED LIST

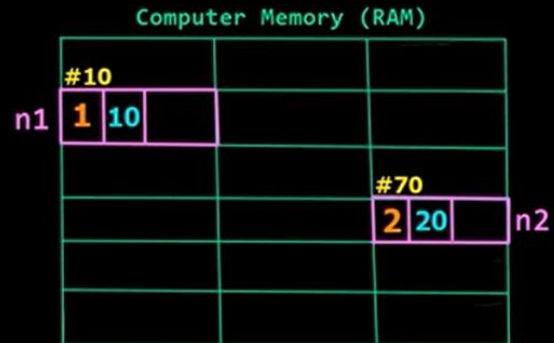
Creating a node -

```
class Node{
public:
    int key;
    int data;
    Node* next;
    Node()
    {
        key = 0;
        data = 0;
        next = NULL;
    }
    Node(int k,int d)
    {
        key = k;
        data=d;
        next=NULL;
    }
};
```

Node object

K D next

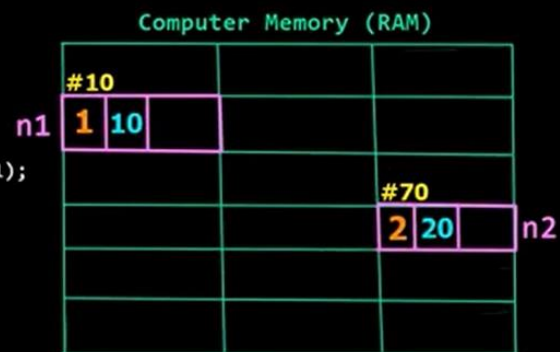
```
int main()
{
    Node n1(1,10);
    Node n2(2,20);
}
```



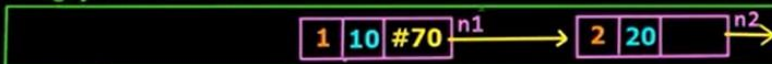
```
class SinglyLinkedList
{
public:
    Node *head;
    SinglyLinkedList(Node* h)
    void prependNode(Node* n)
    void appendNode(Node* n)
    void insertNode(key)
    void deleteNode(key)
    void updateNode(key)
}
```

```
int main()
{
    Node n1(1,10);
    Node n2(2,20);
    Node n3(3,30);

    SinglyLinkedList s(&n1);
    s.appendNode(&n2);
    s.prependNode(&n3);
}
```



SinglyLinkedList s



=====

EXAMPLE CODE

UNDERSTANDING OF WHAT IS LINKED LIST & SINGLY LINKED LIST

c++ objects and classes
pointers in c++
dynamic memory allocation

DISCLAIMER: There are different ways to implement linked list (and other ds) so this is the way
I'll go ahead

Singly Linked List Operations

appendNode()
prependNode()
insertNodeAfter()
deleteNodeByKey()
updateNodeByKey()
print()

```
#include <iostream>
using namespace std;
```

```
class Node{
public:
    int key;
    int data;
    Node* next;
```

```
Node() {
    key = 0;
    data = 0;
    next = NULL;
```

```
Node(int k,int d){
    key = k;
    data = d;
```

```

class SinglyLinkedList{
public:
    Node* head;

    SinglyLinkedList(){
        head = NULL;
    }

    SinglyLinkedList(Node *n){
        head = n;
    }

// 1. check if node exist using key value
    Node* nodeExist(int k){
        Node* temp = NULL;
        Node* ptr = head;
        while(ptr!=NULL){
            if(ptr->key==k){
                temp = ptr;
            }
            ptr = ptr->next;
        }
        return temp;
    }

// 2. Append a anode to the list   i.e adding new node at the end of list
    void appendNode(Node *n){
        if(nodeExist(n->key)!=NULL){
            cout<<"node already exist with key value: "<<n->key<<". append another node with different
y value"<<endl;
        }else{
            if(head==NULL){
                head = n;
                cout<<"node appended"<<endl;
            }else{
                Node* ptr = head;
                while(ptr->next!=NULL){ ptr = ptr->next;

```



```

        ptr->next=n;
        cout<<"node appended"<<endl;
    }
}
}

//3. prepend node: adding new node at the beginnning i.e = we have to make new added node head
de
void prependNode(Node *n) {
    if (nodeExist(n->key) != NULL) {
        cout<<"node already exist with key value: "<<n->key<<". prepend another node with different
y value"<<endl;
    }else{
        n->next=head;
        head = n;
    }
}

//4. inserting a node after a particular node in the list//
void insertNodeAfter(int k,Node* n ) {
    Node* ptr = nodeExist(k);
    if(ptr==NULL) {
        cout<<"node with key value: "<<k<<" does not exist"<<endl;
    }else{
        if (nodeExist(n->key) != NULL) {
            cout<<"node already exist with key value: "<<n->key<<". prepend another node with different
y value"<<endl;
        }else{
            n->next = ptr->next;
            ptr->next =n;
            cout<<"noder inserted"<<endl;
        }
    }
}

//5. Deleting a node with a particular key//
void deleteNodeByKey(int k) {
    }
}

```



```

}else if(head!=NULL) {
    if(head->key==k) {
        head = head->next;
        cout<<"node unlinked with key value: "<<k<<endl;
    }else{
        Node* temp = NULL;
        Node* prevptr = head;
        Node* currentptr = head->next;
        while(currentptr!=NULL) {
            if(currentptr->key==k) {
                temp = currentptr;
                currentptr =NULL;
            } else{
                prevptr = prevptr->next;
                currentptr = currentptr->next;
            }
        }
        if(temp!=NULL) {
            prevptr->next = temp->next;
            cout<<"node unlinked with key value: "<<k<<endl;
        }else{
            cout<<"node with key value: "<<k<<" does not exist"<<endl;
        }
    }
}
}

```

//6. update node by key//

```
void updateNodeByKey(int k, int d)
```

```

{
    Node* ptr = nodeExist(k);
    if(ptr!=NULL) {
        ptr->data = d;
        cout<<"Node data updated successfully"<<endl;
    }
}

```

//7. printing

```
if(head==NULL){ cout<<"singly linked list
```

```
already empty can't delete  
anything"<<endl;  
void printlist(){
```



```

}else{
    cout<<endl<<"singly linked list values: ";
    Node* temp = head;

    while(temp!=NULL){
        cout<<" ("<<temp->key<<","<<temp->data<<") -->";
    }
}

t main()

SinglyLinkedList s;
int option;
int key1, k1, data1;
do{
    cout<<"\n what operatoin do you want to perform. select from options. Enter 0 to
it"<<endl;
    cout<<"1. appendNode() "<<endl;
    cout<<"2. prependNode() "<<endl;
    cout<<"3. insertNodeAfter() "<<endl;
    cout<<"4. deleteNodeByKey() "<<endl;
    cout<<"5. updateNodeByKey() "<<endl;
    cout<<"6. print() "<<endl;
    cout<<"7. clear screen"<<endl;

    cin>>option;
    Node* n1 = new Node();

    switch(option){
        case 0:
            break;

        case 1:
            cout<<"append Node operation \nEnter key & data of the node to append"<<endl;
            cin>>key1;
            cin>>data1;

```

list"<<e



```

        s.appendNode(key1, data1);
break;

case 2:
    cout<<"prepend node operation \n enter key and data to prepend"<<endl;
    cin>>key1;
    cin>>data1;
    n1->key=key1;
    n1->data=data1;
    s.prependNode(n1);
break;

case3:
    cout<<"insert node after operation \nEnter key of existing node after which you want to
sert this new node: "<<endl;
    cin>>key1;
    cout<<"insert node after operation \nEnter key of existing node after which you want to
sert this new node: "<<endl;
    cin>>k1;
    cout<<"enter key & data of new node first: "<<endl;
    cin>>key1;
    cin>>data1;
    n1->key=key1;
    n1->data=data1;
    s.insertNodeAfter(k1, n1);
break;

case 4:
    cout<<"delete node by key operation. \nEnter key of node to be deleted:"<<endl;
    cin>>k1;
    s.deleteNodeByKey(k1);
break;

case 5:
    cout<<"update node by key operation. \nEnter key and new data to be updated"<<endl;
    cin>>key1;
    cin>>data1;
    s.updateNodeByKey(key1, data1);

```

>data=



```

break;

case 6:
    cout<<"display linked list operation"<<endl;
    s.printlist();
    break;

case 7:
    system("cls");
    break;

default:
    cout<<"enter proper option number"<<endl;
}
}while(option!=0);
return 0;

```



Types of Linked List -

D = Data P = Previous N = Next

1) Singly Linked List	2) Doubly Linked List	3) Circular Linked List
(head)	(head)	(head)

DOUBLY LINKED LIST



Operations

Operations of Linked List -

1. Traversing a linked list. (head) 	2. Append a new node (to the end) of a list (head) 	3. Prepend a new node (to start) of list (head)
4. Inserting a new node to a specific position in the list (head) 	5. Deleting a node from the list (head) 	6. Updating a node in the list (head)

Creating a node -

```
class Node{
public:
    int key;
    int data;
    Node* next;
    Node* previous;
```

```
Node()
{
    key=0;
    data=0;
    next=NULL;
    previous=NULL;
}
Node(int k,int d)
{
    key = k;
    data= d;
    next=NULL;
    previous=NULL;
}
};
```

Node Obj



```
int main()
{
    Node n1(1,10);
    Node n2(2,20);
```

Computer Memory (RAM)

n1	#10		
	null	1	10 null
n2	#70		
	null	2	20 null

Node n1

prev* key data next*

Doubly linked list example

```
=====
```

```
doublyLinkedList(Node* n) {
```

```
    
#include<iostream>
using namespace std;
class Node{
public:
    int key;
    int data;
    Node* next;
    Node* previous;

    Node() {
        key =0;
        data = 0;
        next = NULL;
        previous = NULL;
    }

    Node(int k, int d){
        key = k;
        data = d;
    }
}
```

```
class doublyLinkedList{
public:
    Node* head;

    doublyLinkedList(){
        head = NULL;
    }
}
```

```

        head =n;
    }

Node* NodeExists(int k){
    Node* temp =NULL;
    Node* ptr = head;

    while(ptr!=NULL){
        if(ptr->key==k){
            temp = ptr;
        }
        ptr = ptr->next;
    }
    return temp;
}

// 2. append a node to the list//
void appendNode(Node* n){
    if(NodeExists(n->key)!=NULL){
        cout<<"Node already exists with key value"<<n->key<<"append another node with different key
value"<<endl;
    }else{
        if(head==NULL){
            head = n;
            cout<<"node appended as head node"<<endl;
        }else{
            Node* ptr = head;
            while(ptr->next!=NULL){
                ptr = ptr->next;
            }
            ptr->next = n;
            n->previous = ptr;
            cout<<"node appended"<<endl;
        }
    }
}

```

```
/3. prepend node i.e= attaching a node at the start //
```

```
void prependNode(Node* n){
```

```
    if(NodeExists(n->key) !=NULL) {
```



```

y"<<endl;

else{
    if(head==NULL) {
        head = n;
        cout<<"node prepended as head node"<<endl;
    }else{

head->previous=n;
n->next = head;
head = n;
cout<<"node prepended"<<endl;
    }

/4. insert a node after a particular node//
void insertNodeAfter(int k, Node *n){
    Node* ptr = NodeExists(k);
    if(ptr==NULL) {
        cout<<"No node exists with key value: "<<k<<endl;
    }else{
        if(NodeExists(n->key) !=NULL) {
            cout<<"Node already exists with key value: "<<n->key<<"append another node with
different key value"<<endl;

        }else{
            cout<<"inserting"<<endl;
            Node *nextNode = ptr->next;

            // inserting at the end//
            if(nextNode==NULL) {
                ptr->next = n;
                n->previous = ptr;
                cout<<"node inserted at the end"<<endl;
            }
            // inserting in between
        }else{
            cout<<"Node already exists with key value"<<n->key<<"prepend another node with different

```

```
n->next = nextNode;
```



```

        ptr->next = n;
        cout<<"node inserted in between"<<endl;
    }
}

/ 5. delete node by unique key. basically de-link not delete
void deleteNodeByKey(int k){
    Node* ptr = NodeExists(k);
    if(ptr==NULL){
        cout<<"No node exists with key value: "<<k<<endl;
    }else{
        if(head==NULL){
            cout<<"List is empty. can't delete"<<endl;
        }else if(head!=NULL){
            if(head->key==k){
                head = head->next;
                cout<<"Node Unlinked with key values: "<<k<<endl;
            }else{
                Node *nextNode = ptr->next;
                Node *prevNode = ptr->previous;

                // deleting at the end //
                if(nextNode==NULL){
                    prevNode->next = NULL;
                    cout<<"node deleted at the end"<<endl;
                }
                // deleting in between //
            }else{
                prevNode->next = nextNode;
                nextNode->previous = prevNode;
                cout<<"node deleted in between"<<endl;
            }
        }
    }

    nextNode->previous=n; n-

```

```
>previous = ptr;
```




```

void updateNodeByKey(int k, int d){
    Node* ptr = NodeExists(k);
    if(ptr!=NULL){
        ptr->data=d;
        cout<<"node data updated successfully "<<endl;
    }else{
        cout<<"No node exists with key value: "<<k<<endl;
    }
}

/ 7. printing
void printList(){
    if(head==NULL){
        cout<<"no node in doubly linked list"<<endl;
    }else{
        cout<<endl<<"Doubly linked list values: ";
        Node* temp = head;

        while(temp != NULL){
            cout << "(" << temp->key << "," << temp->data << ")"<< "-->";
            temp = temp->next;
        }
    }
}

int main()

doublyLinkedList obj;

int option;
int key1,k1,data1;
do{
    cout<<"\nWhat operation do you want to perform. select from options"<<endl;

    cout<<"2. prependNode() "<<endl;
    cout<<"3. insertNodeAfter() "<<endl;
    cout<<"4. deleteNodeByKey() "<<endl;
/ 6. update node //

```

```
cout<<"1. appendNode()"<<endl;
```



```

cin>>option;
Node* n1 = new Node();
//Node n1 //

switch(option) {
    case 0:
        break;

    case 1:
        cout<<"append node operation \nEnter key and data to append"<<endl;
        cin>>key1;
        cin>>data1;
        n1->key=key1;
        n1->data=data1;
        obj.appendNode(n1);
        break;

    case 2:
        cout<<"prepend node operation \nEnter key and data to prepend"<<endl;
        cin>>key1;
        cin>>data1;
        n1->key=key1;
        n1->data=data1;
        obj.prependNode(n1);
        break;

    case 3:
        cout<<"insert node after operation \nEnter key of existing node after which you want to
sert this new node: "<<endl;
        cin>>key1;
        cout<<"Enter key and data of new node to be inserted"<<endl;
        cin>>k1;
        cin>>data1;
        n1->key=key1;
        n1->data=data1;
        obj.insertNodeAfter(k1,n1);
        cout<<"5. updateNodeByKey()"<<endl;

```

```
cout<<"6. print()"<<endl; cout<<"7.  
    clrscr screen"<<endl;  
  
break;
```



```

cin>>key1;
obj.deleteNodeByKey(key1);
break;

case 5:
cout<<"update node by key operation \nEnter key of node to be updated"<<endl;
cin>>key1;
cin>>data1;
obj.updateNodeByKey(key1,data1);
break;

case 6:
obj.printList();
break;

case 7:
system("cls");
break;

default:
cout<<"Invalid choice. \nEnter proper option number"<<endl;
}
}while(option!=0);

return 0;

```

```

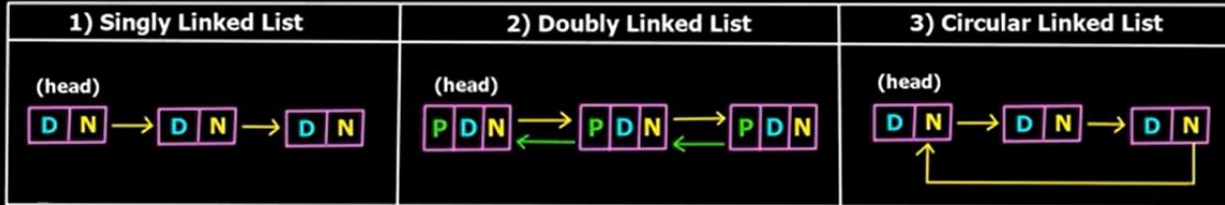
case 4: cout<<"delete node by key operation \nEnter key of node to be
deleted"<<

```

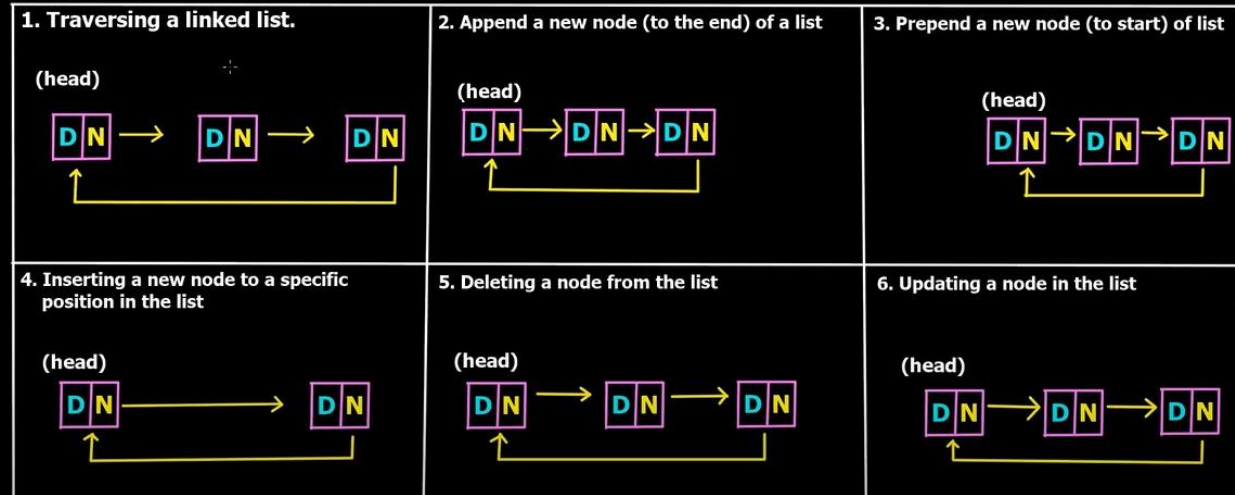
CIRCULAR LINKED LIST

Types of Linked List -

D = Data P = Previous N = Next



Operations of Linked List -



Creating a node -

```
class Node{
public:
    int key;
    int data;
    Node* next;

    Node()
    {
        key=0;
        data=0;
        next=NULL;
    }

    Node(int k,int d)
    {
        key = k;
        data= d;
        next=NULL;
    }
};
```

Node Obj



```
int main()
{
    Node n1(1,10);
    Node n2(2,20);
}
```

Computer Memory (RAM)



Creating a node -

Node Obj

K	D	next
---	---	------

Append_node algorithm/pseudo code

1. Get the new node.
2. Check if node exists with same key
 - 2a. if true, abort
 - 2b. if false continue
- 3a. if head == NULL then append at start
- 3b. else traverse to the end of the list
4. Append new node at the end

HOW TO APPEND A NODE

Creating a node -

Node Obj

K	D	next
---	---	------

Append_node algorithm/pseudo code

1. Get the new node.
2. Check if node exists with same key
 - 2a. if true, abort
 - 2b. if false continue
- 3a. if head == NULL then append at start
- 3b. else traverse to the end of the list
4. Append new node at the end

```
int main()
```

```
{
    Node n1(1,10);
    Node n2(2,20);
    Node n3(3,30);
    Node n4(4,40);

    CircularLinkedList c
    c.appendNode(&n1);
    c.appendNode(&n2);
    c.prependNode(&n3);
    c.insertNodeAfter(1,&n4);
    c.deleteNode(3);
    c.updateNode(4,55);
}
```

Computer Memory (RAM)

			head
#10			
n1	1	10	-
			#70
#45			2 20 -
n3	3	30	-
			#90
			4 40 -
			n4

head* ptr

--	--	--	--	--	--	--	--

How to prepend node

Prepend node algorithm/pseudo code

1. Get the new Node.
2. Check if node exists with same key
 - 2a. if true, abort
 - 2b. if false continue
3. Assign head value to a temporary pointer - ptr
4. Use ptr to traverse to end of list
5. Use ptr to access next pointer of last node to point to this newly passed node
6. point next pointer of n to head ptr.
7. change head ptr to n

How to insert node after

Insert node after algorithm

1. Get the new Node & the key of the node in the list after which you want to link this new node.
2. Check if node exists with same key as new node
 - 2a. if true, abort
 - 2b. if false continue
3. Check if node exists with the key entered by user
 - 3a. if false, abort
 - 3b. if true continue
4. access node N after which you want to append new node new_N
 - 4a. If node to be inserted at the end then
new_node->next = head
previous_node->next = new_node
 - 4b. if node to be inserted in between then
new_node->next = previous_node->next
previous_node->next = new_node

HOW TO DELETE NODE BY KEY

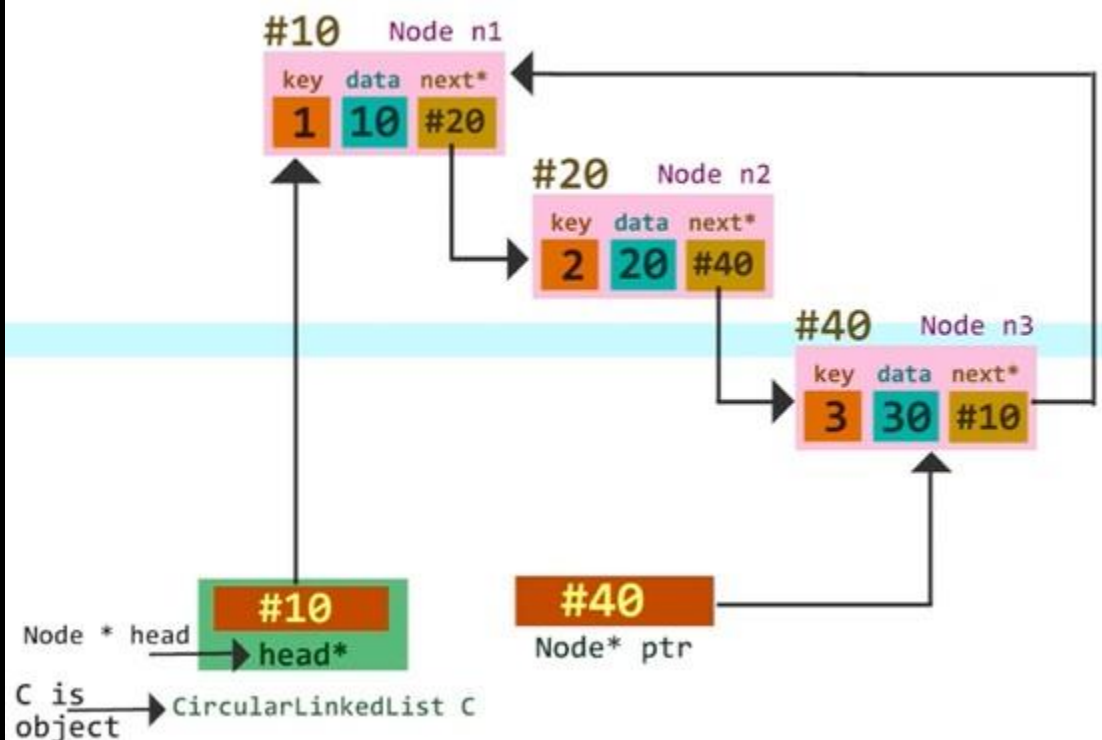
Creating a node -

Node Obj

K	D	next
---	---	------

Delete node by key algorithm -

1. Get the key of node you want to delete.
2. Check if node exists with same key
 - 2a. if false, abort
 - 2b. if true continue
- 3a. if head == NULL then list empty
- 3b. if head != NULL && key matches head node then
head = head->next
last_node->next = head
- 3c. if head != NULL && key doesn't match head node then
Traverse to the node with the key.(ptr)
- 4a. if node at the end then
previousNode->next = head
- 4b. if node in between then
prevNode->next = ptr->next



=====

EXAMPLE CODE

```
#include<iostream>

using namespace std;

class Node{
public:
    int key;
    int data;
    Node* next;

    Node(){
        key = 0;
        data = 0;
        next = NULL;
    }

    Node(int k, int d){
        key = k;
        data = d;
    }
};

class circularLinkedList{
public:
    Node* head;

    circularLinkedList(){
        head = NULL;
    }
// add int if error continue in line 28
    int singlyLinkedList(Node* n){
        head = n;
    }

    // 1. check if node exist using key value
    Node* nodeExist(int k){
        Node* temp = NULL;
```

```

Node* ptr = head;

if(ptr==NULL){
    return temp;
}else{
    do{
        if(ptr->key==k){
            temp = ptr;
        }
        ptr=ptr->next;
    }while(ptr!=head);
    return temp;
}

//return temp
}

//2. Append a node to the list
void appendNode(Node *new_node){
    if(nodeExist(new_node->key)!=NULL){
        cout << "Node already exist with key " << new_node->key
<<"append another node with different key value"<<endl;
    }else{
        if(head==NULL){
            head = new_node;
            new_node->next=head;
            cout<<"node appended at first head poition"<<endl;
        }else{
            Node* ptr=head;

```

```

        while(ptr->next!=head)
        {
            ptr = ptr->next;
        }
        ptr->next = new_node;
        new_node->next = head;
        cout<<"node appended"<<endl;
    }
}

```

//3. prepend a node to the list i.e= adding a node at the beginning if the list

```

void prependNode(Node* new_node){
    if(nodeExist(new_node->key)!=NULL){
        cout<<"node already exist with key value:
"<<new_node->key<<"append another node with different key
value"<<endl;
    }else{
        if(head==NULL){
            head = new_node;
            new_node->next = head;
            cout<<"node prepended as head node"<<endl;
        }else{
            Node* ptr = head;
            while(ptr->next==head){
                ptr = ptr->next;
            }
            ptr->next=new_node;
            new_node->next = head;
            head = new_node;
            cout<<"node appended"<<endl;
        }
    }
}

```

```

    }
}

// 4. insert a node after a particular node in the list
void insertNodeAfter(int k, Node* new_node){
    Node *ptr = nodeExist(k);
    if(ptr==NULL){
        cout<<"node with key "<<k<<" does not exist"<<endl;
    }else{
        if(nodeExist(new_node->key) !=NULL){
            cout<<"node already exist with key value:
"<<new_node->key<<"append another node with different key
value"<<endl;
        }else{
            if(ptr->next==head){
                new_node->next=head; ptr->
                next=new_node; cout<<"node inserted
at the end"<<endl; }else{ new_node->
                next = ptr->next ptr->next =
                new_node;
                cout<<"node inserted in between"<<endl;
            }
        }
    }
}

// 5. delete node by a unique key
void deleteNodeByKey(int k){

```

```

Node* ptr = nodeExist(k);
if(ptr==NULL) {
    cout<<"node with key "<<k<<" does not exist"<<endl;
}else{
    if(head==NULL) {
        cout<<"list is empty. Can't delete anything"<<endl;
    }else if(ptr==head){
        if(head->next==NULL) {
            head = NULL;
            cout<<"head node unlinked list is empty"<<endl;
        }else{
            Node* ptr = head;
            while(ptr->next!=head) {
                ptr = ptr->next;
            }
            ptr->next = head->next;
            head = head->next;
            cout<<"node unlinked with key value:
"<<k<<endl;
        }
    }else{
        Node* temp=NULL;
        Node* prevptr =head;
        Node* currentptr = head->next;
        while(currentptr!=NULL) {
            if (currentptr->key==k) {
                prevptr->next = currentptr->next;
                currentptr->next = NULL;
            }
            prevptr = currentptr;
            currentptr = currentptr->next;
        }
    }
}
}

```

```

        }else{ prevptr =prevptr-
        >next; }
    }
    prevptr->next=temp->next;
    cout<<"node unlinked with key value: "<<k<<endl;
}
}
}

```

// 6. update node

```
void updateNode(int k, int d){
```

```
    Node* ptr = nodeExist(k);
```

```
    if(ptr!=NULL){
```

```
        ptr->data = d;
```

```
        cout<<"node updated with key value: "<<k<<endl;
```

```
    }else{
```

```
        cout<<"node with key "<<k<<" does not exist"<<endl;
```

```
    }
```

```
}
```

// 7th printing

```
void printList() {
```

```
    if (head == NULL) {
```

```
        cout << "No Nodes in Circular Linked List";
```

```
    } else {
```

```
        cout << endl << "head address : " << head << endl;
```

```
        cout << "Circular Linked List Values : " << endl;
```

```
        Node * temp = head;
```

```

        do {
            cout << "(" << temp -> key << "," << temp -> data << "," <<
temp -> next << ")" --> ";
            temp = temp -> next;
        } while (temp != head);
    }
}

```

```

int main()
{
    circularLinkedList obj;
    int option;
    int key1, k1, data1;
    do {
        cout << "\nWhat operation do you want to perform? Select Option
number. Enter 0 to exit." << endl;
        cout << "1. appendNode()" << endl;
        cout << "2. prependNode()" << endl;
        cout << "3. insertNodeAfter()" << endl;
        cout << "4. deleteNodeByKey()" << endl;
        cout << "5. updateNodeByKey()" << endl;
        cout << "6. print()" << endl;
        cout << "7. Clear Screen" << endl << endl;

        cin >> option;
        Node * n1 = new Node();
        //Node n1;

        switch (option) {

```



```

        case 0:
            break;

        case 1:
            cout << "Append Node Operation \nEnter key & data of the Node
to be Appended" << endl;
            cin >> key1;
            cin >> data1;
            n1 -> key = key1;
            n1 -> data = data1;
            obj.appendNode(n1);
            //cout<<n1.key<<" = "<<n1.data<<endl;
            break;

        case 2:
            cout << "Prepend Node Operation \nEnter key & data of the
Node to be Prependded" << endl;
            cin >> key;
            cin >> data1; n1 ->
            key = key; n1
            data = data1;
            obj.prependNode(n1);
            break;

        case 3:
            cout << "Insert Node After Operation \nEnter key of existing
Node after which you want to Insert this New node: " << endl;
            cin >> k1;
            cout << "Enter key & data of the New Node first: " << endl;
            cin >> key1;
            cin >> data1;
            n1 -> key = key1;

```

```
        n1 -> data = data1;

        obj.insertNodeAfter(k1, n1);
        break;

    case 4:

        cout << "Delete Node By Key Operation - \nEnter key of the
Node to be deleted: " << endl;
        cin >> k1;
        obj.deleteNodeByKey(k1);

        break;

    case 5:
        cout << "Update Node By Key Operation - \nEnter key & NEW
data to be updated" << endl;
        cin >> key1;
        cin >> data1;
        obj.updateNode(key1, data1);

        break;

    case 6:
        obj.printList();

        break;

    case 7
```

```
        cout << "Enter Proper Option number " << endl;
    }

} while (option != 0);

return 0;
}
```

```
system("cls")
break default
```



SEARCHING

Linear Search Algorithm -

1. Take input array from user
2. Take element you want to search from user
3. Start from 1st element in array to last
4. if match found
 then -> print message.
 stop process.
 else
 move to next element in array
5. END

0	1	2	3
55	62	7	11

ALGORITHM

Searching Algorithms -

Searching Algorithms are designed to check for an element or retrieve an element from any data structure where it is stored. Based on the type of search operation, these algorithms are generally classified into two categories:

- 1) Sequential Search
- 2) Interval Search

Sequential Search -

In this, the list or array is traversed sequentially and every element is checked.
For example: Linear Search.

0	1	2	3	4
55	62	7	11	173

Interval Search -

These algorithms are specifically designed for searching in sorted data-structures. These type of searching algorithms are much more efficient than Linear Search as they repeatedly target the center of the search structure and divide the search space in half.

For Example: Binary Search.

0	1	2	3	4
7	11	55	62	173

LINEAR SEARCH ALGORITHM

=====
=====

EXAMPLE

```
#include <iostream>

using namespace std;

void linearSearch(int a[], int n){
    int temp = -1;
    for(int i = 0;i<5;i++){
        if(a[i]==n){
            cout<<"element found at location: "<<i<<endl;
            temp = 0;
            break;
        }
    }
    if(temp== -1){
        cout<<"element not found"<<endl;
    }
}

int main(){
    int arr[5];
    cout<<"enter 5 elements of the array"<<endl;
    for(int i=0;i<5;i++){
        cin>>arr[i];
    }
    cout<<"enter an element to search"<<endl;
    int num;
    cin>>num;

    linearSearch(arr,num);

    system("pause");

    return 0;
}
```

BINARY SEARCH ALGORITHM

Interval Search -

These algorithms are specifically designed for searching in sorted data-structures. These type of algorithms are much more efficient than Linear Search as they repeatedly target the center of the structure and divide the search space in half.

For Example: Binary Search.

0	1	2	3	4
7	11	55	62	173

Binary Search Algorithm Working -

1. Search a sorted array by repeatedly dividing the search interval in half.
2. Begin with an interval covering the whole array.
3. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half.
4. Otherwise narrow it to the upper half.
5. Repeatedly check until the value is found or the interval is empty.

```
>> Compare x with the middle element.  
>> If x matches with middle element,  
    we return the mid index.  
>> Else If x is greater than the mid element,  
    then x can only lie in right half subarray  
    after the mid element. So we recur for right half.  
>> Else (x is smaller) recur for the left half.
```

The idea of binary search is to use the information that the array is sorted and reduce the time complexity to $O(\log n)$.

arr[10] = {1 3 12 14 23 34 55 65 75 78}

	1	3	12	14	23	34	55	65	75	78
index →	0	1	2	3	4	5	6	7	8	9
positions										

Binary Search Algorithm -

arr[10] = {1 3 12 14 23 34 55 65 75 78}

	1	3	12	14	23	34	55	65	75	78
index →	0	1	2	3	4	5	6	7	8	9
positions										

1. take input : array, left, right & x
2. START LOOP - while(left<=right)
 - 2.1 mid = left + (right-left)/2
 - 2.2 if(arr[mid]==x) then
 - 2.2.1 return mid
 - 2.3 else if(arr[mid]<x) then
 - 2.3.1 left = mid + 1
 - 2.4 else
 - 2.4.1 right = mid - 1
- END LOOP
3. return -1

Step 1 - Take inputs

array = arr | left = 0 | right = 9 | x =

Step 2 - LOOP While(left<=right)

mid = left + (right-left)/2

Example code

```
while(left<=right){  
    int mid = left + (right - left) / 2;  
    if(arr[mid]==x){  
  
#include <iostream>  
using namespace std;  
  
int binarySearch(int arr[], int left, int right, int x){
```

```

        return mid;
    }else if(arr[mid]<x){
        left = mid + 1;
    }else{
        right = mid - 1;
    }
}
return -1;
}

int main(){

    // int arr[10];
    // cout<<"enter 10 elements of array"<<endl;
    // for(int i =0;i<10;i++){
    //     cin>>arr[i];
    // }
    int num;
    int arr[10];
    int output;

    cout<<"inter 10 integers in ascending order"<<endl;
    for(int i =0;i<10;i++){
        cin>>arr[i];
    }
    cout<<"enter an integer that you want to search"<<endl;
    cin>>num;

    output = binarySearch(arr, 0, 9, num);

    if (output == -1){
        cout<<"no match found"<<endl;
    }else{
        cout<<"match found at index position: "<<output<<endl;
    }
    system("pause");
    return 0;
}

```



```
}
```

```
=====
=====
=====
=====
```

SORTING ALGORITHMS

What is a Sorting Algorithm ?

A Sorting Algorithm is used to rearrange a given array or list DS elements according to a particular pattern(For e.g. Ascending or Descending order).

The comparison operator is used to decide the new order of element in the respective data structure.

Why do we need sorting algorithms ?

1. Efficient sorting is important for optimizing the efficiency of other algorithms (such as search and merge algorithms) which require input data to be in sorted lists.
2. Producing more human readable output.
3. It is easier and faster to locate items in a sorted list than unsorted.

Types of Sorting Algorithms ?

1.Selection Sort 2.Bubble Sort 3.Insertion Sort 4.Merge Sort
5.Quick Sort 6.Heap Sort 7.Radix Sort etc etc etc

SELECTION SORT ALGORITHM

- >> Selection sort is a sorting algorithm, specifically an in-place comparison sort.
- >> It has $O(n^2)$ time complexity, making it inefficient on large lists.
- >> The algorithm divides the input list into two parts: the sublist of items already sorted, which is built up from left to right at the front (left) of the list, and the sublist of items remaining to be sorted that occupy the rest of the list.
- >> Initially, the sorted sublist is empty and the unsorted sublist is the entire input list.
- >> The algorithm proceeds by finding the smallest (or largest, depending on sorting order) element in the unsorted sublist, exchanging (swapping) it with the leftmost unsorted element (putting it in sorted order), and moving the sublist boundaries one element to the right.

```
int arr[5] = {64, 25, 12, 22, 11}
```

0	1	2	3	4
64	25	12	22	11

Applying Selection Sort -

1. Find minimum element & place it at position 0

	0	1	2	3	4
sorted sub-array	11	25	12	22	64
un-sorted sub-array					

2. Find minimum element & place it at position 1

	0	1	2	3	4
sorted sub-array	11	12	25	22	64
un-sorted sub-array					

3. Find minimum element & place it at position 2

	0	1	2	3	4
sorted sub-array	11	12	22	25	64
un-sorted sub-array					

4. Find minimum element & place it at position 3

	0	1	2	3	4
sorted sub-array	11	12	22	25	64
un-sorted sub-array					

Selection Sort Algorithm

arr : array of items
n : size of list

n:	5
i:	
min:	
j:	

```
for(i = 0; i < n - 1; i++)  
/* set current element as minimum */  
    min = i  
  
    /* check the element to be minimum */  
  
    for(j = i+1; j < n; j++)  
        if arr[j] < arr[min] then  
            min = j;  
        end if  
    end for  
  
    /* swap the minimum element & current element*/  
    if(min != i) then  
        swap arr[min] and arr[i]  
    end if  
end for  
int arr[5] = {64, 25, 12, 22, 11}
```

0	1	2	3	4
64	25	12	22	11

EXAMPLE CODE

```
#include <iostream>  
  
using namespace std;  
  
void selectionSort(int arr[]){  
    for(int i = 0; i < 4; i++){ // int i = 0 // 0 is the index  
        position in first iteration it is 0 in second iteration it will be 1  
        int min = i;  
        for(int j = i+1; j < 5; j++){ // comparison happens in the  
            inner for loop  
                if(arr[j] < arr[min]){  
                    min = j;  
                }  
            }  
        }  
    }  
}
```

```
int main() {
```

```
int myarr[5];

cout<<"enter 5 random integers"<<endl;
for(int i =0;i<5;i++){
    cin>>myarr[i];
}

cout<<"unsorted array: "<<endl;
for(int i = 0;i<5;i++){
    cout<<myarr[i]<<" ";
}

cout<<endl;
```

=====

=====

INSERTION SORT ALGORITHM

Insertion Sort Algorithm -

Theory -

- >> Insertion sort is a simple sorting algorithm that works the way we sort playing cards in our hands.
- >> Insertion sort is a simple sorting algorithm that builds the final sorted array (or list) one item at a time.
- >> Efficient for (quite) small data sets, much like other quadratic sorting algorithms
- >> More efficient in practice than most other simple quadratic (i.e., $O(n^2)$) algorithms such as selection sort or bubble sort
- >> **Time Complexity: $O(n^2)$**

```
int arr[5] = {9, 7, 3, 6, 2}
```

0	1	2	3	4
9	7	3	6	2

Insertion Sort Algorithm -

Working -

- Step 1 – Pick next element
- Step 2 – Compare with all elements in the sorted sub-list on the left
- Step 3 – Shift all the elements in the sorted sub-list that is greater than the value to be sorted
- Step 4 – Insert the value
- Step 5 – Repeat until list is sorted

```
int arr[5] = {9, 7, 3, 6, 2}
```

0	1	2	3	4
9	7	3	6	2

Insertion Sort Algorithm

a: array of items
n: size of list

start

1. declare variables - i, key, j

2. loop : 1 to n-1 //outer loop

2.1 key = a[i];

2.2 j = i - 1;

2.3 loop: (j >= 0 && arr[j] > key) // inner loop

2.3.1 arr[j + 1] = arr[j];

2.3.2 j = j - 1;

end loop // inner loop

2.4 arr[j + 1] = key;

end loop // outer loop

stop

```
int arr[5] = {9, 7, 3, 6, 2}
```

0	1	2	3	4
9	7	3	6	2

n:	5
i:	
key:	
j:	

Example code

```
#include <iostream>

using namespace std;

void insertionSort(int arr[]){
    int j = 0;
    int key = 0;

    for(int i = 0; i<5; i++){
        key = arr[i]; // picking the next element
        j = i-1;
        while(j>=0 && arr[j]>key){
            arr[j+1] = arr[j];
            j = j-1;
        }
        arr[j+1] = key;
    }
}

int main(){
    int myarray[5];
    cout<<"enter 5 integers in array in any order"<<endl;
    for(int i=0;i<5;i++){
        cin>>myarray[i];
    }

    cout<<"before sorting : "<<endl;
    for(int i=0;i<5;i++){
        cout<<myarray[i]<<" ";
    }

    insertionSort(myarray);

    cout<<"\nafter sorting : "<<endl;
    for(int i=0;i<5;i++){
        cout<<myarray[i]<<" ";
    }
}
```

```
system("pause");  
return 0;  
}
```

=====

BUBBLE SORT ALGORITHM



Bubble Sort Algorithm -

Theory -

- >> Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.
- >> This algorithm is suitable for small data sets
- >> It is known as bubble sort, because with every complete iteration the largest element bubbles up towards the last place or the highest index just like a water bubble rises up to the water surface.

>> **Time Complexity: $O(n^2)$**

Bubble Sort Algorithm -

Working -

- Step 1 - Starting with the first element(index = 0), compare the current element with the next element of the array.
- Step 2 - If the current element is greater than the next element of the array, swap them.
- Step 3 - If the current element is less than the next element, move to the next element.
- Step 4 - Repeat Step 1 till the list is sorted.

Bubble Sort Algorithm

a: array of items
n: size of list

n:	5
i:	
j:	

start

1. declare variables - i, j

2. loop : (i = 0 to i < n) //outer loop

2.1 loop: (j = 0 to j < n-i-1) // inner loop

2.1.1 if(a[j] > a[j+1]) then

2.1.1.1 swap(a[j] , a[j+1])

end if

end loop // inner loop

end loop // outer loop

stop

EXAMPLE CODE

```
#include <iostream>

using namespace std;

void bubbleSort(int a[]){
    for(int i =0; i<5;i++){
        for(int j=0;j<(5-i-1);j++){
            if (a[j]>a[j+1]){
                int temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
}

int main(){

    int myarray[5];
    int size;
    cout<<"enter 5 integers in any order"<<endl;
    for(int i=0;i<5;i++){
        cin>>myarray[i];
    }

    cout<<"before sorting"<<endl;
    for(int i=0;i<5;i++){
        cout<<myarray[i]<<" ";
    }

    bubbleSort(myarray); // sorting

    cout<<"\nAfter sorting: "<<endl;
    for(int i=0;i<5;i++){
        cout<<myarray[i]<<" ";
    }
}
```

Bubble Sort Algorithm

a: array of items
n: size of list

n:	
i:	
j:	
rounds:	

```
start
1. declare variables - i, j
2. loop :(i = 0 to i < n) //outer loop
    2.1 loop:(j = 0 to j < n-i-1) // inner loop
        2.1.1 if(a[j] > a[j+1]) then
            2.1.1.1 swap(a[j] , a[j+1])
            end if
        end loop // inner loop
    end loop // outer loop
stop
```

```
int arr[5] = {5, 2, 4, 3, 6}
```

0	1	2	3	4
5	2	4	3	6

if list is not empty there is no mechanism to find out if the list is already sorted. This is the reason why we need to find out if the list is already sorted before we start sorting.

OPTIMIZED Bubble Sort Algorithm

a: array of items

n: size of list

start

1. declare variables - i, j

2. loop : (i = 0 to i < n) //outer loop

2.1 flag = false

2.2 loop: (j = 0 to j < n-i-1) // inner loop

2.2.2 if(a[j] > a[j+1]) then

2.2.2.1 flag = true

2.2.2.2 swap(a[j] , a[j+1])

end if

end loop // inner loop

2.3 if (flag == false) then BREAK

end loop // outer loop

stop

int arr[5] = {5, 2, 4, 3, 6}

0	1	2	3	4
5	2	4	3	6

n:	
i:	
j:	
rounds:	
flag:	

EXAMPLE CODE

```
#include <iostream>
using namespace std;

void bubbleSort(int a[]){
    int rounds = 0;
    for(int i =0; i<5;i++){
        rounds++;
        for(int j=0;j<(5-i-1);j++){
            if (a[j]>a[j+1]){
                int temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }

    cout << "\nNumber Of Rounds: " << rounds << endl;
}
```

```

void optimizedbubbleSort(int a[]){

    int rounds = 0;
    for(int i =0; i<5;i++){
        rounds++;
        int flag = false; // inside first outer loop
        for(int j=0;j<(5-i-1);j++){
            if (a[j]>a[j+1]){
                flag = true;
                int temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
        if (flag==false){ // outside inner for loop
            break;
        }
    }
    cout << "\nNumber Of Rounds: " << rounds << endl;
}

int main(){

    int myarray[5];
    cout<<"enter 5 integers in any order"<<endl;
    for(int i=0;i<5;i++){
        cin>>myarray[i];
    }

    cout<<"before sorting"<<endl;
    for(int i=0;i<5;i++){
        cout<<myarray[i]<<" ";
    }

    bubbleSort(myarray); // sorting

```

```
cout<<"\nAfter sorting: "<<endl;
for(int i=0;i<5;i++){
    cout<<myarray[i]<<" ";
}

optimzedbubbleSort(myarray); // sorting
```

```
cout<<"\nAfter sorting: "<<endl;
for(int i=0;i<5;i++){
    cout<<myarray[i]<<" ";
}

system("pause");
return 0;
```

BIG O NOTATION

WHAT IS ALGORITHM ANALYSIS

Simple Definition –

- Algorithm analysis is a **study** to provides theoretical **estimation** for the required **resources** of an algorithm to solve a specific computational problem. i.e. calculating **efficiency**.

Generally, the **efficiency** an algorithm is related to the input length (number of steps), known as **time complexity**, or volume of memory, known as **space complexity**.

WHY DO WE NEED ALGORITHM ANALYSIS

- Knowing efficiency of an algorithm is very vital on **mission critical** tasks.
- Generally there are **multiple** approaches/method/algorithms to solve one problem statement. Algorithm analysis is performed to figure out which is the **better/optimum** approaches/method/algorithms out of the options.

What does a **BETTER** Algorithm mean ?

- Faster ? (Less execution time) – **Time Complexity**
- Less Memory ? – **Space Complexity**
- Easy to read ?
- Less Line of Code ?
- Less Hw/Sw needs ?

*Note: Algorithm Analysis **does not give you accurate/exact values**(time, space etc), however it gives **estimates** which can be used to study the **behavior** of the algorithm.

What is **Asymptotic** Algorithm Analysis ?

- Definition: In mathematical analysis, **asymptotic** analysis of algorithm is a method of defining the **mathematical boundaries** of its **run-time** performance.
- Using the asymptotic analysis, we can easily estimate about the average case, best case and worst case scenario of an algorithm.

Simple words: It is used to mathematically calculate the running time of any operation inside an algorithm.

Asymptotic Algorithm analysis is to estimate the **time complexity** function for arbitrarily large input.

Time Complexity : is a computational way to show **how(behavior)** **runtime** of a program increases as the size of its input increases.

BIG O NOTATION

>> The notation $O(n)$ is the formal/mathematical way to express the upper bound (worst case) of an algorithm's running time.

>> It measures the worst case time complexity or the longest amount of time an algorithm can possibly take to complete.

Following is a list of some common asymptotic notations –

>> constant time - $O(1)$ >> linear - $O(n)$

>> logarithmic - $O(\log n)$ >> quadratic - $O(n^2)$

>> cubic - $O(n^3)$

Big O notation is used to describe how the time or space complexity of an algorithm grows as the size of the input increases.

- ❖ $O(1)$ Constant space or time complexity means that the input size does not affect the algorithm's performance in an algorithmically independent manner.
- ❖ $O(n)$ Linear time or space complexity means that the time or space grows directly with the size of the input (e.g., searching for an element in a list).
- ❖ $O(n^2)$ Quadratic time or space complexity means that the time or space grows as the square of the input size (e.g., checking for duplicates in a list).

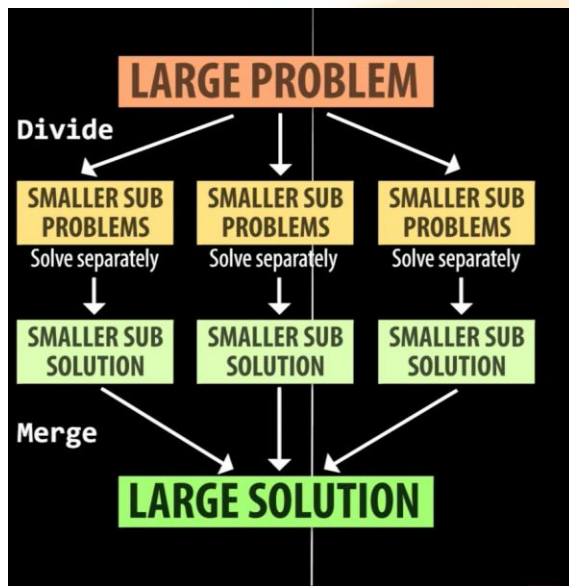
Big O helps us understand how efficient an algorithm is as the problem size grows larger.

SORTING ALGORITHMS

MERGE SORT ALGORITHM

Theory -

- >> Merge Sort is a **Divide** and **Conquer** algorithm.
- >> It divides input array in two halves, calls itself for the two halves(recursively) and then merges the two sorted halves.
- >> A separate merge() function is used for merging two halves.
- >> Merge sort is one of the most efficient sorting algorithms.
- >> **Time Complexity: $O(n\log(n))$**



DIVIDE AND CONQUER

PUACP

Merge Sort Algorithm (Working)

```
mergeSort(arr[], l, r)
{
```

Step 1 - Find the middle point to divide the array into two halves:

$$\text{middle } m = (l+r)/2$$

Division

Step 2 - Call mergeSort for first half :

`mergeSort(arr, l, m)`

Step 3 - Call mergeSort for second half:

`mergeSort(arr, m+1, r)`

Recursion

Step 4 - Merge the two halves sorted in step 2 and 3:

`merge(arr, l, m, r)`

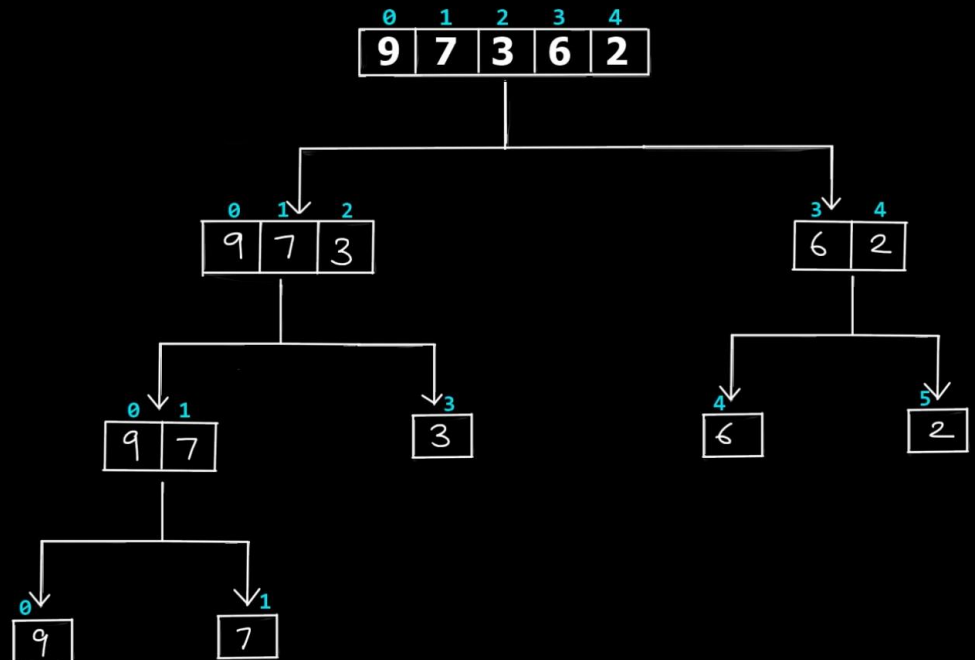
Merging

```
}
```

```
int arr[5] = {9, 7, 3, 6, 2}
```

0	1	2	3	4
9	7	3	6	2

```
mergeSort(arr[], l, r)
{
    if(l<r)
    {
        1. m = (l+r)/2
        2. mergeSort(arr, l, m)
        3. mergeSort(arr, m+1, r)
        4. merge(arr, l, m, r)
    }
}
```



EXAMPLE CODE

```
#include <iostream>
```



```

    }
}

// merge sort function
void mergeSort(int arr[],int l, int r){
    if(l<r){
        int m = (l+r)/2;
        mergeSort(arr, 0, m);
        mergeSort(arr,m+1,r);
        merge(arr,l,m,r);
    }
}

int main(){
    cout<<"enter five elements in the array"<<endl;
    int myarr[5];
    for(int i=0;i<5;i++){
        cin>>myarr[i];
    }
    cout<<"before merge sort"<<endl;
    for(int i=0;i<5;i++){
        cout<<myarr[i]<<" ";
    }

    //merge sort function
    mergeSort(myarr, 0, 4);

    cout<<"\nafter merge sort"<<endl;
    for(int i=0;i<5;i++){
        cout<<myarr[i]<<" ";
    }
    system("pause");
    return 0;
}

```

```

=====
=====

```

QUICK SORT SORTING ALGORITHM

Quick Sort Algorithm -

- >> Quick Sort is a **Divide** and **Conquer** algorithm.
- >> It picks an element as **pivot** and **partitions** the given array around the picked pivot.
- >> There are many different versions of quickSort that pick pivot in different ways.
 1. Always pick first element as pivot.
 2. Always pick last element as pivot (our example)
 3. Pick a random element as pivot.
- >> Quick sort is considered as in in-place sorting algorithm. It reduces the space complexity and removes the use of the auxiliary array that is used in merge sort.
- >> Quick sort is one of the most efficient sorting algorithms.
 - >> Time Complexity : $\Theta(n \log(n))$
 - >> Space Complexity : $O(\log(n))$

```
int arr[5] = { 7, 9, 3, 2, 6 }
```

0	1	2	3	4
7	9	3	2	6

Quick Sort Algorithm (Working)

```
QuickSort(arr[], s, e)
```

s -> start index
e -> end index

```
{
```

Step 1 - Find partition index (till start index < end index)

```
p = Partition(arr[], s, e)
```

Division

Step 2 - Call QuickSort for first half :

```
QuickSort(arr[], s, p-1)
```

Step 3 - Call QuickSort for second half:

```
QuickSort(arr[], p+1, e)
```

Recursion

```
}
```

```
int arr[5] = { 7, 9, 3, 2, 6 }
```

0	1	2	3	4
7	9	3	2	6

```
QuickSort(arr[], s, e)
```

```
{  
  if(s<e)  
  {  
    p = Partition(arr[],s,e)  
    QuickSort(arr[], s, (p-1))  
    QuickSort(arr[], (p+1), e)  
  }  
}
```

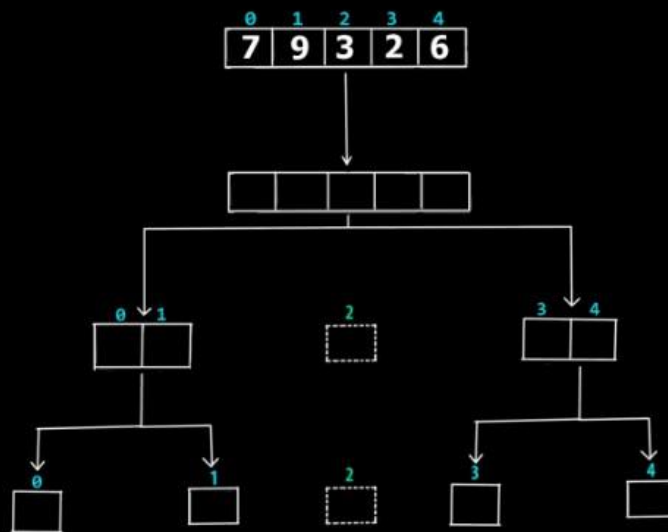
PUACP

```

QuickSort(arr[], s, e)
{
    if(s < e)
    {
        p = Partition(arr[], s, e)
        QuickSort(arr[], s, (p-1))
        QuickSort(arr[], (p+1), e)
    }
}

Partition(arr[], s, e)
{
    pivot = arr[e]
    pIndex = s
    for (i = s to e-1)
    {
        if(arr[i] <= pivot)
        {
            swap(arr[i], arr[pIndex])
            pIndex++
        }
    }
    swap(arr[e], arr[pIndex])
    return pIndex
}

```



EXAMPLE CODE

```

#include <iostream>
using namespace std;

int Partition(int arr[], int s, int e){
    int pivot = arr[e];
    int pIndex = s;
    for(int i=s;i<e;i++){
        if(arr[i]<pivot){
            int temp = arr[i];
            arr[i] = arr[pIndex];
            arr[pIndex] = temp;
            pIndex++;
        }
    }
    int temp = arr[e];
    arr[e] = arr[pIndex];
    arr[pIndex] = temp;
}

```



```

system("pause");
return 0;
}

```

BUCKET / DIGITAL / RADIX SORT ALGORITHM

Radix Sort Algorithm -

- >> Radix sort is a non-comparative sorting algorithm.
- >> It avoids comparison by creating and distributing elements into buckets according to their radix.
- >> For elements with more than one significant digit, this bucketing process is repeated for each digit, while preserving the ordering of the prior step, until all digits have been considered. For this reason, radix sort has also been called bucket sort and digital sort.
- >> Typically Radix sort uses counting sort as a subroutine to sort.
- >> Radix sort has linear time complexity which is better than $O(n \log n)$ of comparative sorting algorithms.

>> **Time Complexity :** $O(d(n+k))$

>> **Space Complexity :** $O(n+k)$

where n is the number of elements in input array and k is the range of input.

```
int arr[8] = {170, 45, 75, 90, 802, 24, 2, 66}
```

0	1	2	3	4	5	6	7
170	45	75	90	802	24	2	66

Radix Sort Algorithm - (Working)

Working -

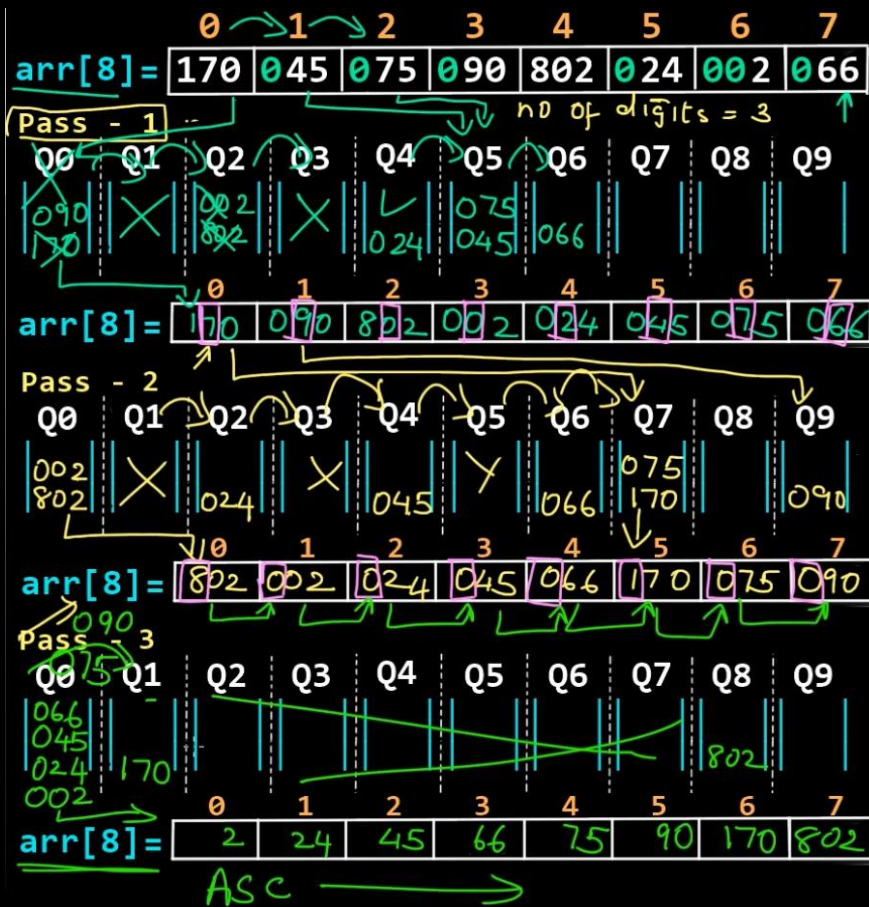
- Step 1 - Take input array & find the MAX number in the array
- Step 2 - Define 10 queues each representing a bucket for each digit from 0 to 9.
- Step 3 - Consider the least significant digit of each number in the list which is to be sorted.
- Step 4 - Insert each number into their respective queue based on the least significant digit.
- Step 5 - Group all the numbers from queue 0 to queue 9 in the order they have inserted into their respective queues.
- Step 6 - Repeat from step 3 based on the next least significant digit.
- Step 7 - Repeat from step 2 until all the numbers are grouped based on the most significant digit.

```
int arr[8] = {170, 45, 75, 90, 802, 24, 2, 66}
```

0	1	2	3	4	5	6	7
170	45	75	90	802	24	2	66



DRY RUN



Pseudocode -

RadixSort(arr, size)

{

1. take arr[size]

2. Get max from the arr

2.1 $m = \text{GetMax}(\text{arr}, \text{size})$

3. Do counting sort for every digit.

3.1 for (int div = 1; m/div > 0; div *= 10)

3.1.1 CountingSort(arr, size, div)

}

GetMax(arr, size)

{

1. max = arr[0]

2. for (int i = 1; i < size; i++)

2.1 if (arr[i] > max)

2.1.1 max = arr[i]

3. return max

}

EXAMPLECODE

```
#include<iostream>
using namespace std;

// A utility function to get maximum value in arr[]
int getMax(int arr[], int size)
{
    int max = arr[0];
    for (int i = 1; i < size; i++)
        if (arr[i] > max)
            max = arr[i];
    return max;
}

void CountingSort(int arr[], int size, int div)
{
    int output[size];
    int count[10] = {0};

    for (int i = 0; i < size; i++)
        count[ (arr[i]/div)%10 ]++;

    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for (int i = size - 1; i >= 0; i--)
    {
        output[count[ (arr[i]/div)%10 ] - 1] = arr[i];
        count[ (arr[i]/div)%10 ]--;
    }

    for (int i = 0; i < size; i++)
        arr[i] = output[i];
}

void RadixSort(int arr[], int size)
```

```

{
    int m = getMax(arr, size);
    for (int div = 1; m/div > 0; div *= 10)
        CountingSort(arr, size, div);
}

int main()
{
    int size;
    cout<<"Enter the size of the array: "<<endl;
    cin>>size;
    int arr[size];
    cout<<"Enter "<<size<<" integers in any order"<<endl;
    for(int i=0;i<size;i++)
    {
        cin>>arr[i];
    }
    cout<<endl<<"Before Sorting: "<<endl;
    for(int i=0;i<size;i++)
    {
        cout<<arr[i]<<" ";
    }

    RadixSort(arr, size);

    cout<<endl<<"After Sorting: "<<endl;
    for(int i=0;i<size;i++)
    {
        cout<<arr[i]<<" ";
    }

    return 0;
}

```

```

=====
=====

```

SHELL SORT ALGORITHM

Shell Sort Algorithm -

- >> Shell Sort is an in-place comparison sort.
- >> It is mainly a variation of sorting by exchange (bubble sort) or sorting by insertion (insertion sort)
- >> This algorithm avoids large shifts as in case of insertion sort, if the smaller value is to the far right and has to be moved to the far left.
- >> The idea of shellSort is to allow exchange of far (at a particular gap/interval) items fast.
- >> This spacing is termed as interval or gap.
- >> ShellSort is more efficient compared to Insertion Sort or Bubble sort specially when -
 1. Smaller value elements are towards the end of the array/list
 2. Large sized array/list
 3. Efficiency depends on how we select the GAP/interval

Time Complexity -

1. Best Case - $\Omega(n \log(n))$
2. Worst Case - $O(n^2)$
3. Average Case - $\theta(n \log(n)^2)$

Space Complexity

(Auxiliary Space) - $O(1)$

EXAMPLE CODE

```
#include<iostream>

using namespace std;

void ShellSort(int arr[], int n)
{
    for(int gap=n/2;gap>0;gap /= 2)
    {
        for(int j = gap;j<n;j+=1)
        {
            int temp = arr[j];
            int i=0;

            for(i=j; (i>=gap) && (arr[i-gap]>temp); i-=gap)
            {
                arr[i] = arr[i-gap];
            }
            arr[i] = temp;
        }
    }
}
```

```

int main()
{
    int n;
    cout<<"Enter the size of the array: "<<endl;
    cin>>n;
    int arr1[n],arr2[n];

    cout<<"Enter "<<n<<" integers in any order"<<endl;
    for(int i=0;i<n;i++)
    {
        cin>>arr1[i];
        arr2[i]=arr1[i];
    }

    cout<<endl<<"Before Sorting: "<<endl;
    for(int i=0;i<n;i++)
    {
        cout<<arr1[i]<<" ";
    }

    cout<<endl<<endl<<"SHELL SORT "<<endl;

    ShellSort(arr1, n); // SHELL SORT CALLED HERE

    cout<<endl<<"After Sorting: "<<endl;
    for(int i=0;i<n;i++)
    {
        cout<<arr1[i]<<" ";
    }

    return 0;
}

```

=====

STACK

INFIX / PREFIX /POSTFIX

What is an Expression?

An Expression is a combination of symbols that can be numbers (constants), variables, operations, symbols of grouping and other punctuation written in a specific format/way.

Operand -

An entity on which operation is performed (numbers, variables, constants)

Operator -

An entity that decides which operation to be performed (+ is for addition)

Infix Expression -

<operand> <operator> <operand>

EXAMPLE: $A + B$

A & B are operand (on which operation is performed)

And (+

Prefix Expression -

<operator> <operand> <operand>

EXAMPLE

$a + b \rightarrow \text{infix}$
 $+ab \rightarrow \text{Prefix}$

Postfix Expression -

<operand> <operand> <operator>

Infix, Prefix, Postfix

Depending on how the expression is written, we can classify it into 3 categories -

- 1) **Prefix** - An expression is called the prefix expression if the operator appears in the expression before the operands.
Simply of the form (operator operand1 operand2). E.g. +AB
- 2) **Infix** - An expression is called the infix expression if the operator appears in the expression in between the operands.
Simply of the form (operand1 operator operand2). E.g. A+B
- 3) **Postfix** - An expression is called the postfix expression if the operator appears in the expression after the operands.
Simply of the form (operand1 operand2 operator). E.g. AB+

Prefix	Infix	Postfix
+23	2+3	23+
-xy	x-y	xy-
+a*bc	a+b*c	abc*+



Order of Operations / Operator Precedence -

- 1) **Parentheses** - {}, [], ()
- 2) **Exponents**(Right to Left) - A^B , 2^3^4
- 3) **Multiplication & Division**(Left to Right) - $A*B/C$
- 4) **Addition & Subtraction**(Left to Right) - $A + B - C$

Associativity -

Associativity describes the rule where operators with the same precedence appear in an expression. For example, in expression $a + b - c$, both + and - have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators.

Why do we need 3 different expression formats ?

- >> Infix expressions are human readable but not efficient for machine reading
- >> Prefix and Postfix do not need the concept of precedence & associativity hence it becomes highly efficient to parse expressions in prefix or postfix formats.

Order of Operations / Operator Precedence -

- 1) Parentheses - {}, [], ()
- 2) Exponents(Right to Left) - A^B , 2^3^4
- 3) Multiplication & Division(Left to Right) - $A*B/C$
- 4) Addition & Subtraction(Left to Right) - $A + B - C$

Associativity -

Associativity describes the rule where operators with the same precedence appear in an expression. For example, in expression $a + b - c$, both $+$ and $-$ have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators.



INFIX TO PREFIX CONVERSION

Steps to convert INFIX to PREFIX expression -

- 1) Apply appropriate brackets/parenthesis to expressions by following operator precedence & associativity rules.
- 2) Start by converting inner most sub expressions to prefix & name it with temporary placeholder.
- 3) Repeat step 2 on other larger sub expressions till the full expression is solved. Re-substitute the temporary placeholders with their original values

Order of Operations / Operator Precedence -

- 1) Parentheses - {}, [], ()
- 2) Exponents(Right to Left) - A^B , 2^3^4
- 3) Multiplication & Division(Left to Right) - $A*B/C$
- 4) Addition & Subtraction(Left to Right) - $A + B - C$

Associativity -

Associativity describes the rule where operators with the same precedence appear in an expression. For example, in expression $a + b - c$, both $+$ and $-$ have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators.

Q2) Convert the following infix expression to prefix expression -

$$\begin{aligned}
 & a + (b * c) - (d/e) \\
 & \downarrow \quad \downarrow \quad \downarrow \\
 & [a + x_1] - [x_2] = - + a x_1 / d e \\
 & \rightarrow [a + (b * c)] - [(d/e)] \\
 & \rightarrow [a + (*bc)] - [(/de)] \quad \begin{matrix} *bc \Rightarrow x_1 \\ /de \Rightarrow x_2 \end{matrix} \\
 & \rightarrow [a + x_1] - [x_2] \quad \begin{matrix} +a x_1 \Rightarrow x_3 \\ x_3 - x_2 \Rightarrow (-x_3 x_2) \end{matrix} \\
 & \rightarrow [+a x_1] - [x_2] \\
 & \rightarrow x_3 - x_2 = (-x_3 x_2)
 \end{aligned}$$

Q3) Convert the following infix expression to prefix expression -

$$\begin{aligned}
 & (A + B^C) * D + E \\
 & \rightarrow (A + (^BC)) * D + E \quad \wedge BC = x_1 \\
 & \rightarrow (A + x_1) * D + E \\
 & \rightarrow (+A x_1) * D + E \quad +A x_1 = x_2 \\
 & \rightarrow (x_2 * D) + E \\
 & \rightarrow (* x_2 D) + E \quad \begin{matrix} op1 \\ op2 \end{matrix} \\
 & \rightarrow + * x_2 D = + * + A x_1 D = \\
 & \quad \quad \quad = + * + A ^ B C D
 \end{aligned}$$

PREFIX INFIX TO CONVERSION

Steps to convert PREFIX to INFIX expression

- 1) Start from left to right till you get first instance of 1 operator followed by 2 operands.
- 2) Convert that to infix notation & assume/substitute it with a temporary variable
- 3) Proceed forward from there to right or start again if end of expression is arrived.
- 4) Perform step 2 again.

Order of Operations / Operator Precedence -

- 1) Parentheses - {}, [], ()
- 2) Exponents (Right to Left) - A^B , 2^{3^4}
- 3) Multiplication & Division (Left to Right) - $A*B/C$
- 4) Addition & Subtraction (Left to Right) - $A + B - C$

Associativity -

Associativity describes the rule where operators with the same precedence appear in an expression. For example, in expression $a + b - c$, both $+$ and $-$ have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators.

Q1) Convert the follow prefix expression to infix expression -

$$\begin{aligned} & +(-ab)c \\ \Rightarrow & + (a-b)c \quad (a-b) = x_1 \\ & + x_1 c \\ & (x_1 + c) \\ & [a - b + c] \end{aligned}$$

Q2) Convert the follow prefix expression to infix expression -

$$\begin{aligned} & - + a (* b c) / d e \\ \hookrightarrow & - + a (b * c) (d / e) \quad b * c = x_1 \\ & \quad \quad \quad d / e = x_2 \\ & - + a x_1 x_2 \\ & - (a + x_1) x_2 \quad (a + x_1) = x_3 \\ & = x_3 - x_2 \\ & = a + x_1 - x_2 \\ & = a + b * c - d / e \quad \text{INFIX} \end{aligned}$$

Q3) Convert the follow prefix expression to infix expression -

$+ * + A \wedge B C D E$

$+ * + A (B^C) D E \quad B^C = x_1$

$+ * + A x_1 D E$

$+ * (A + x_1) D E \quad A + x_1 = x_2$

$+ * x_2 D E$

$+ (x_2 * D) E$

$\rightarrow x_2 * D + E$

$A + x_1 * D + E$

$A + B^C * D + E$ INFIX



INFIX TO POSTFIX CONVERSION

Steps to convert INFIX to POSTFIX Expression -

- 1) Apply appropriate brackets/parenthesis to expressions by following operator precedence & associativity rules.
- 2) Start by converting inner most sub expressions to postfix & name it with temporary placeholder.
- 3) Repeat step 2 on other larger sub expressions till the full expression is solved. Re-substitute the temporary placeholders with their original values

Order of Operations / Operator Precedence -

- 1) Parentheses - {}, [], ()
- 2) Exponents(Right to Left) - A^B , 2^3^4
- 3) Multiplication & Division(Left to Right) - $A*B/C$
- 4) Addition & Subtraction(Left to Right) - $A + B - C$

Associativity -

Associativity describes the rule where operators with the same precedence appear in an expression. For example, in expression $a + b - c$, both $+$ and $-$ have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators.

Q1) Convert the following infix expression to postfix expression -

$$\begin{aligned}
 & \boxed{(a - b) + c} \\
 & \downarrow \\
 & (a \ b -) + c \\
 & \quad \boxed{a \ b - = x_1} \\
 & \quad \downarrow \\
 & \quad x_1 + c \\
 & \quad \downarrow \\
 & \quad x_1 \ c + \\
 & \quad \downarrow \\
 & = \boxed{a \ b - \ c +} \text{ POSTFIX}
 \end{aligned}$$

Q2) Convert the following infix expression to postfix expression -

$$a + b * c - d/e$$

$$= A + B * C - D/E$$

$$= A + (B * C) - (D/E) = A + (BC *) - (DE /)$$

$$\text{Let } BC * = x_1, \quad DE / = x_2$$

$$= A + x_1 - x_2 = (A + x_1) - x_2 = (A x_1 +) - x_2$$

$$\text{Let } A x_1 + = x_3$$

$$= X_3 - X_2$$
$$= x_3 x_2$$

NOW

SUBSTIT

UTING

VALUES

BACK = A

$$X_1 + X_2 =$$

Q3) Convert the following infix expression to postfix expression -

$$(A+B^{\wedge}C)*D+E$$
$$(C)^* D + E = [A + (B^* C)^*]^* + E = (A + B^* C)^* + E = (A + B^* C)^*$$
$$\therefore BC^A = \frac{1}{(1 + \frac{A}{100})^n} [C + \frac{E}{A} (1 + \frac{A}{100})^n - E] \therefore A \times \frac{100}{A} + = X$$
$$E = (X_2 - 1) + \dots + X_3 + E \quad : \quad 2 \square = 3 = \langle 3 \rangle + \text{NO}$$
$$D^+E + ABC + D^-E + \dots = ABC + D^-E + \dots = \text{FINAL}$$

POSTFIX TO INFIX

Q1) Convert the following postfix expression to infix expression -

$$a \quad b \quad - \quad c \quad +$$

Q2) Convert the following postfix expression to infix expression -

a b c * + d e / -

Q3) Convert the following postfix expression to infix expression -

$A \ B \ C^{\wedge} \ + \ D \ * \ E \ +$

INFIX TO POSTFIX USING STACK

Rules for Infix to postfix using stack DS -

- 1) Scan Expression from Left to Right
- 2) Print **OPERANDs** as they arrive
- 3) If **OPERATOR** arrives & Stack is empty, push this operator onto the stack
- 4) IF incoming **OPERATOR** has **HIGHER** precedence than the **TOP** of the Stack, push it on stack
- 5) IF incoming **OPERATOR** has **LOWER** precedence than the **TOP** of the Stack, then **POP** and print the **TOP**. Then test the incoming operator against the **NEW TOP** of stack.
- 6) IF incoming **OPERATOR** has **EQUAL** precedence with **TOP** of Stack, use **ASSOCIATIVITY** Rules.
- 7) For **ASSOCIATIVITY** of LEFT to RIGHT - **POP** and print the **TOP** of stack, then **PUSH** the incoming **OPERATOR**
- 8) For **ASSOCIATIVITY** of RIGHT to LEFT - **PUSH** incoming **OPERATOR** on stack.
- 9) At the end of Expression, **POP** & print all **OPERATORs** from the stack

PUACP